

Yulu

May 25, 2025

1 Business Case: Yulu - Hypothesis Testing

1.1 About Yulu

Yulu is India's leading micro-mobility service provider, which offers unique vehicles for the daily commute. Starting off as a mission to eliminate traffic congestion in India, Yulu provides the safest commute solution through a user-friendly mobile app to enable shared, solo and sustainable commuting.

Yulu zones are located at all the appropriate locations (including metro stations, bus stands, office spaces, residential areas, corporate offices, etc) to make those first and last miles smooth, affordable, and convenient!

Yulu has recently suffered considerable dips in its revenues. They have contracted a consulting company to understand the factors on which the demand for these shared electric cycles depends. Specifically, they want to understand the factors affecting the demand for these shared electric cycles in the Indian market.

1.2 Problem Statement

The company wants to know:

- Which variables are significant in predicting the demand for shared electric cycles in the Indian market?
- How well those variables describe the electric cycle demands

Importing Libraries

```
[452]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import datetime as dt
import scipy.stats as spy
```

Reading the dataset

```
[453]: df = pd.read_csv("bike_sharing.csv")
df
```

```
[453]:
```

	datetime	season	holiday	workingday	weather	temp \
0	2011-01-01 00:00:00	1	0	0	1	9.84
1	2011-01-01 01:00:00	1	0	0	1	9.02
2	2011-01-01 02:00:00	1	0	0	1	9.02
3	2011-01-01 03:00:00	1	0	0	1	9.84
4	2011-01-01 04:00:00	1	0	0	1	9.84
...	...	...	...	...	...	...
10881	2012-12-19 19:00:00	4	0	1	1	15.58
10882	2012-12-19 20:00:00	4	0	1	1	14.76
10883	2012-12-19 21:00:00	4	0	1	1	13.94
10884	2012-12-19 22:00:00	4	0	1	1	13.94
10885	2012-12-19 23:00:00	4	0	1	1	13.12

	atemp	humidity	windspeed	casual	registered	count
0	14.395	81	0.0000	3	13	16
1	13.635	80	0.0000	8	32	40
2	13.635	80	0.0000	5	27	32
3	14.395	75	0.0000	3	10	13
4	14.395	75	0.0000	0	1	1
...	...	...	...	...	...	...
10881	19.695	50	26.0027	7	329	336
10882	17.425	57	15.0013	10	231	241
10883	15.910	61	15.0013	4	164	168
10884	17.425	61	6.0032	12	117	129
10885	16.665	66	8.9981	4	84	88

[10886 rows x 12 columns]

Shape of the dataset

```
[454]: df.shape
```

```
[454]: (10886, 12)
```

Columns in the Dataset

```
[455]: df.columns
```

```
[455]: Index(['datetime', 'season', 'holiday', 'workingday', 'weather', 'temp',
          'atemp', 'humidity', 'windspeed', 'casual', 'registered', 'count'],
          dtype='object')
```

Basic information about the values present in the dataset

```
[456]: df.head(10)
```

```
[456]:
```

	datetime	season	holiday	workingday	weather	temp	atemp \
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635

3	2011-01-01 03:00:00	1	0	0	1	9.84	14.395
4	2011-01-01 04:00:00	1	0	0	1	9.84	14.395
5	2011-01-01 05:00:00	1	0	0	2	9.84	12.880
6	2011-01-01 06:00:00	1	0	0	1	9.02	13.635
7	2011-01-01 07:00:00	1	0	0	1	8.20	12.880
8	2011-01-01 08:00:00	1	0	0	1	9.84	14.395
9	2011-01-01 09:00:00	1	0	0	1	13.12	17.425

	humidity	windspeed	casual	registered	count
0	81	0.0000	3	13	16
1	80	0.0000	8	32	40
2	80	0.0000	5	27	32
3	75	0.0000	3	10	13
4	75	0.0000	0	1	1
5	75	6.0032	0	1	1
6	80	0.0000	2	0	2
7	86	0.0000	1	2	3
8	75	0.0000	1	7	8
9	76	0.0000	8	6	14

```
[457]: df.tail(10)
```

```
[457]:
```

	datetime	season	holiday	workingday	weather	temp	\
10876	2012-12-19 14:00:00	4	0	1	1	17.22	
10877	2012-12-19 15:00:00	4	0	1	1	17.22	
10878	2012-12-19 16:00:00	4	0	1	1	17.22	
10879	2012-12-19 17:00:00	4	0	1	1	16.40	
10880	2012-12-19 18:00:00	4	0	1	1	15.58	
10881	2012-12-19 19:00:00	4	0	1	1	15.58	
10882	2012-12-19 20:00:00	4	0	1	1	14.76	
10883	2012-12-19 21:00:00	4	0	1	1	13.94	
10884	2012-12-19 22:00:00	4	0	1	1	13.94	
10885	2012-12-19 23:00:00	4	0	1	1	13.12	

	atemp	humidity	windspeed	casual	registered	count
10876	21.210	50	12.9980	33	185	218
10877	21.210	50	19.0012	28	209	237
10878	21.210	50	23.9994	37	297	334
10879	20.455	50	26.0027	26	536	562
10880	19.695	50	23.9994	23	546	569
10881	19.695	50	26.0027	7	329	336
10882	17.425	57	15.0013	10	231	241
10883	15.910	61	15.0013	4	164	168
10884	17.425	61	6.0032	12	117	129
10885	16.665	66	8.9981	4	84	88

Column Profiling:

- datetime: datetime
- season: season (1: spring, 2: summer, 3: fall, 4: winter)
- workingday: if day is neither weekend nor holiday is 1, otherwise is 0.
- weather:
 - 1: Clear, Few clouds, partly cloudy, partly cloudy
 - 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist
 - 3: Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds
 - 4: Heavy Rain + Ice Pallets + Thunderstorm + Mist, Snow + Fog
- temp: temperature in Celsius
- atemp: feeling temperature in Celsius
- humidity: humidity
- windspeed: wind speed
- casual: count of casual users
- registered: count of registered users
- count: count of total rental bikes including both casual and registered

Is there any null value in the dataset ?

```
[458]: np.any(df.isna())
```

```
[458]: np.False_
```

Is there any duplicated values in the dataset ?

```
[459]: np.any(df.duplicated())
```

```
[459]: np.False_
```

Datatype of the columns

```
[460]: df.dtypes
```

```
[460]: datetime      object
season            int64
holiday           int64
workingday        int64
weather           int64
temp             float64
atemp            float64
humidity          int64
windspeed        float64
casual           int64
```

```
registered      int64
count           int64
dtype: object
```

Converting the datatype of datetime column from object to datetime

```
[461]: df['datetime'] = pd.to_datetime(df['datetime'])
```

What is the time period for which the data is given ?

```
[462]: df['datetime'].min()
```

```
[462]: Timestamp('2011-01-01 00:00:00')
```

```
[463]: df['datetime'].max()
```

```
[463]: Timestamp('2012-12-19 23:00:00')
```

```
[464]: df['datetime'].max() - df['datetime'].min()
```

```
[464]: Timedelta('718 days 23:00:00')
```

```
[465]: df['day'] = df['datetime'].dt.day_name()
```

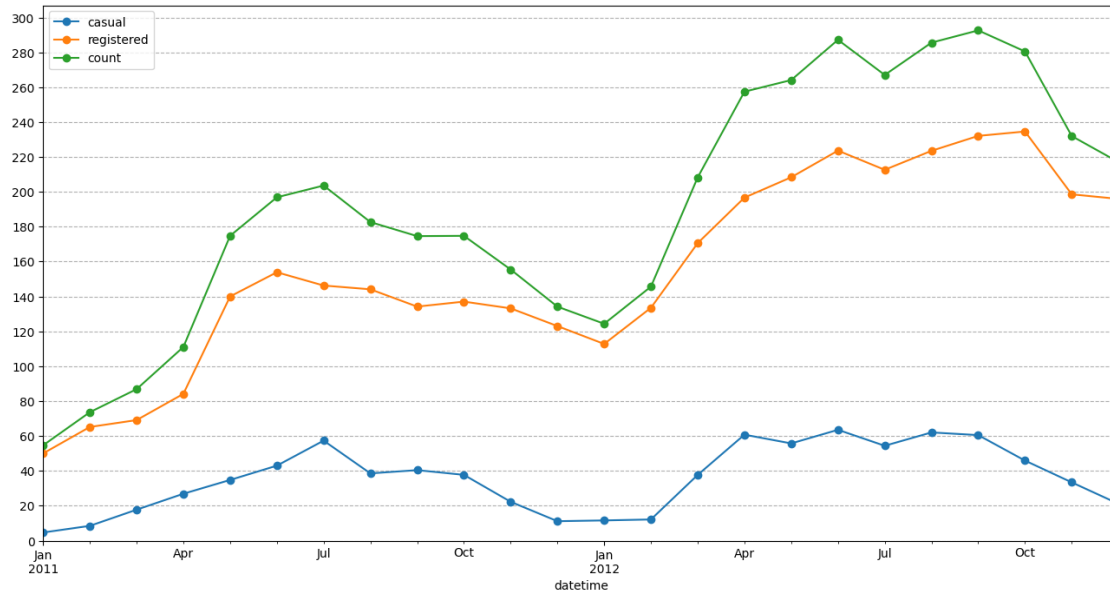
```
[466]: df.set_index('datetime', inplace = True)
```

Slicing Data by Time

```
[467]: plt.figure(figsize = (16, 8))

df.resample('ME')['casual'].mean().plot(kind = 'line', legend = 'casual',
    ↪marker = 'o')
df.resample('ME')['registered'].mean().plot(kind = 'line', legend =
    ↪'registered', marker = 'o')
df.resample('ME')['count'].mean().plot(kind = 'line', legend = 'count', marker
    ↪= 'o')

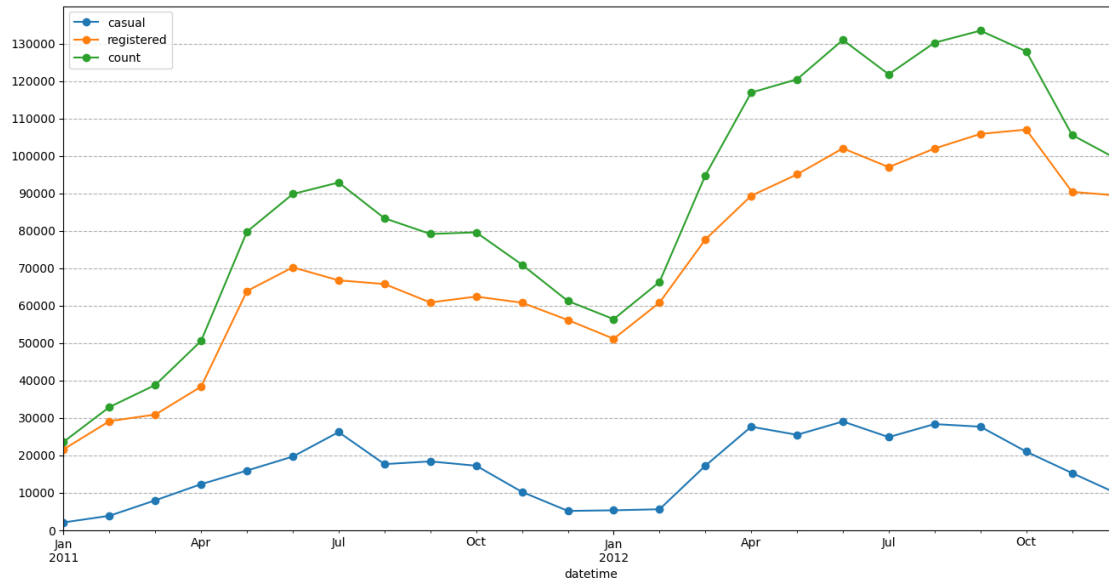
plt.grid(axis = 'y', linestyle = '--')
plt.yticks(np.arange(0, 301, 20))
plt.ylim(0,)
plt.show()
```



```
[468]: plt.figure(figsize = (16, 8))

df.resample('ME')['casual'].sum().plot(kind = 'line', legend = 'casual', marker = 'o')
df.resample('ME')['registered'].sum().plot(kind = 'line', legend = 'registered', marker = 'o')
df.resample('ME')['count'].sum().plot(kind = 'line', legend = 'count', marker = 'o')

plt.grid(axis = 'y', linestyle = '--')
plt.yticks(np.arange(0, 130001, 10000))
plt.ylim(0,)
plt.show()
```



want to know if there is an increase in the average hourly count of rental bikes from the year 2011 to 2012

```
[469]: df1 = df.resample('Y')['count'].mean().to_frame().reset_index()

df1['prev_count'] = df1['count'].shift(1)

# Calculating the growth percentage of 'count' with respect to the 'count' of
# previous year
df1['growth_percent'] = (df1['count'] - df1['prev_count']) * 100 /
df1['prev_count']
df1
```

```
[469]:
```

	datetime	count	prev_count	growth_percent
0	2011-12-31	144.223349	NaN	NaN
1	2012-12-31	238.560944	144.223349	65.410764

- This data suggests that there was substantial growth in the count of the variable over the course of one year.
- The mean total hourly count of rental bikes is 144 for the year 2011 and 239 for the year 2012. An annual growth rate of 65.41 % can be seen in the demand of electric vehicles on an hourly basis.

It indicates positive growth and potentially a successful outcome or increasing demand for the variable being measured.

```
[470]: df.reset_index(inplace = True)
```

How does the average hourly count of rental bikes varies for different month ?

```
[471]: df1 = df.groupby(by = df['datetime'].dt.month)['count'].mean().reset_index()
df1.rename(columns = {'datetime' : 'month'}, inplace = True)

df1['prev_count'] = df1['count'].shift(1)

# Calculating the growth percentage of 'count' with respect to the 'count' of
↳previous month
df1['growth_percent'] = (df1['count'] - df1['prev_count']) * 100 /
↳df1['prev_count']
df1.set_index('month', inplace = True)
df1
```

```
[471]:
```

	count	prev_count	growth_percent
month			
1	90.366516	NaN	NaN
2	110.003330	90.366516	21.730188
3	148.169811	110.003330	34.695751
4	184.160616	148.169811	24.290241
5	219.459430	184.160616	19.167406
6	242.031798	219.459430	10.285440
7	235.325658	242.031798	-2.770768
8	234.118421	235.325658	-0.513007
9	233.805281	234.118421	-0.133753
10	227.699232	233.805281	-2.611596
11	193.677278	227.699232	-14.941620
12	175.614035	193.677278	-9.326465

- The count of rental bikes shows an increasing trend from January to March, with a significant growth rate of 34.70% between February and March.
- The growth rate starts to stabilize from April to June, with a relatively smaller growth rate.
- From July to September, there is a slight decrease in the count of rental bikes, with negative growth rates.
- The count further declines from October to December, with the largest drop observed between October and November (-14.94%).

```
[472]: # The resulting plot visualizes the average hourly distribution of the count of
↳rental bikes for each
# month, allowing for comparison and identification of any patterns or trends
↳throughout the year.
plt.figure(figsize = (12, 6))

plt.title("The average hourly distribution of count of rental bikes across
↳different months")

# Grouping the DataFrame by the month and calculating the mean of the 'count'
↳column for each month.
```



```

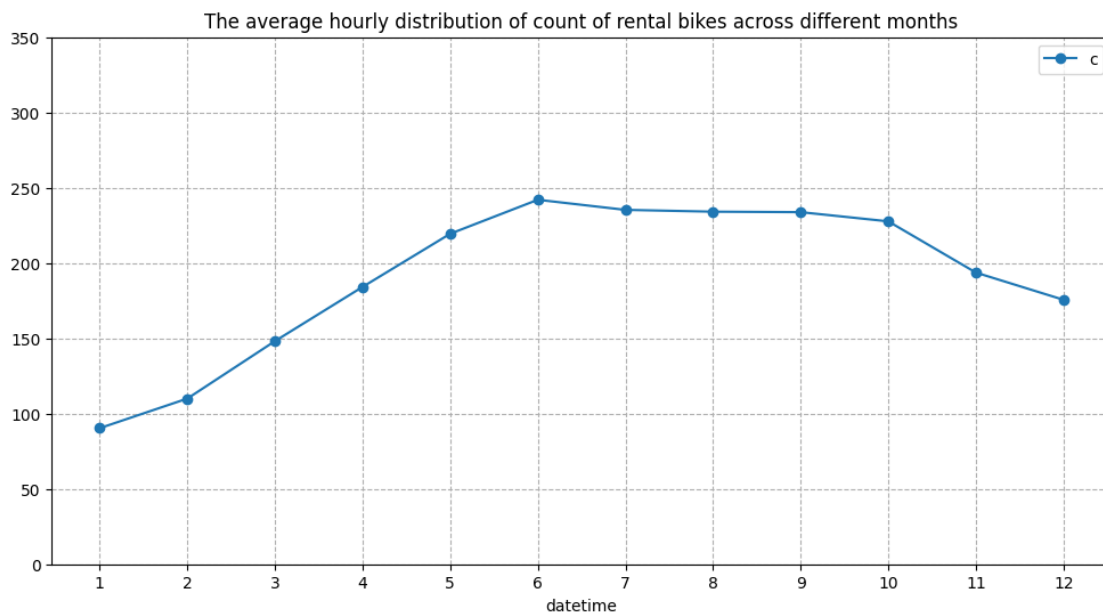
# Plotting the line graph using markers ('o') to represent the average count
↳ per month.
df.groupby(by = df['datetime'].dt.month)['count'].mean().plot(kind = 'line',
↳ marker = 'o')

plt.ylim(0,)
plt.xticks(np.arange(1, 13))
plt.legend('count')
plt.yticks(np.arange(0, 400, 50))

plt.grid(axis = 'both', linestyle = '--')
plt.plot()

```

[472]: []



- The average hourly count of rental bikes is the highest in the month of June followed by July and August.
- The average hourly count of rental bikes is the lowest in the month of January followed by February and March.

Overall, these trends suggest a seasonal pattern in the count of rental bikes, with higher demand during the spring and summer months, a slight decline in the fall, and a further decrease in the winter months. It could be useful for the rental bike company to consider these patterns for resource allocation, marketing strategies, and operational planning throughout the year.

What is the distribution of average count of rental bikes on an hourly basis in a single day ?

```
[473]: # Grouping the DataFrame by the hour
df1 = df.groupby(by = df['datetime'].dt.hour)['count'].mean().reset_index()
df1.rename(columns = {'datetime' : 'hour'}, inplace = True)

# Create a new column 'prev_count' by shifting the 'count' column one position
↳up
    # to compare the previous hour's count with the current hour's count
df1['prev_count'] = df1['count'].shift(1)

# Calculating the growth percentage of 'count' with respect to the 'count' of
↳previous hour
df1['growth_percent'] = (df1['count'] - df1['prev_count']) * 100 /
↳df1['prev_count']
df1.set_index('hour', inplace = True)
df1
```

```
[473]:
```

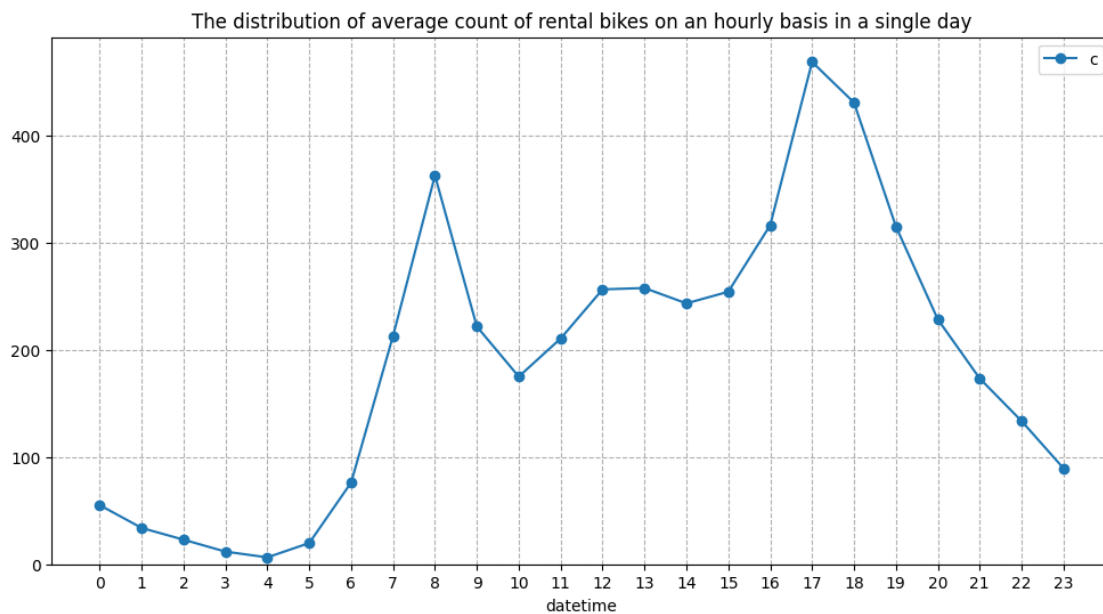
	count	prev_count	growth_percent
hour			
0	55.138462	NaN	NaN
1	33.859031	55.138462	-38.592718
2	22.899554	33.859031	-32.367959
3	11.757506	22.899554	-48.656179
4	6.407240	11.757506	-45.505110
5	19.767699	6.407240	208.521293
6	76.259341	19.767699	285.777526
7	213.116484	76.259341	179.462793
8	362.769231	213.116484	70.221104
9	221.780220	362.769231	-38.864655
10	175.092308	221.780220	-21.051432
11	210.674725	175.092308	20.322091
12	256.508772	210.674725	21.755835
13	257.787281	256.508772	0.498427
14	243.442982	257.787281	-5.564393
15	254.298246	243.442982	4.459058
16	316.372807	254.298246	24.410141
17	468.765351	316.372807	48.168661
18	430.859649	468.765351	-8.086285
19	315.278509	430.859649	-26.825705
20	228.517544	315.278509	-27.518833
21	173.370614	228.517544	-24.132471
22	133.576754	173.370614	-22.953059
23	89.508772	133.576754	-32.990757

- During the early morning hours (hours 0 to 5), there is a significant decrease in the count, with negative growth percentages ranging from -38.59% to -48.66%.
- However, starting from hour 5, there is a sudden increase in count, with a sharp positive growth percentage of 208.52% observed from hour 4 to hour 5.

- The count continues to rise significantly until reaching its peak at hour 17, with a growth percentage of 48.17% compared to the previous hour.
- After hour 17, there is a gradual decrease in count, with negative growth percentages ranging from -8.08% to -32.99% during the late evening and nighttime hours.

```
[474]: plt.figure(figsize = (12, 6))
plt.title("The distribution of average count of rental bikes on an hourly basis,
in a single day")
df.groupby(by = df['datetime'].dt.hour)['count'].mean().plot(kind = 'line',
marker = 'o')
plt.ylim(0,)
plt.xticks(np.arange(0, 24))
plt.legend('count')
plt.grid(axis = 'both', linestyle = '--')
plt.plot()
```

[474]: []



- The average count of rental bikes is the highest at 5 PM followed by 6 PM and 8 AM of the day.
- The average count of rental bikes is the lowest at 4 AM followed by 3 AM and 5 AM of the day.

These patterns indicate that there is a distinct fluctuation in count throughout the day, with low counts during early morning hours, a sudden increase in the morning, a peak count in the afternoon, and a gradual decline in the evening and nighttime.

Basic Information about the Dataset

```
[475]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 13 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   datetime        10886 non-null  datetime64[ns]
 1   season          10886 non-null  int64
 2   holiday         10886 non-null  int64
 3   workingday      10886 non-null  int64
 4   weather         10886 non-null  int64
 5   temp           10886 non-null  float64
 6   atemp          10886 non-null  float64
 7   humidity       10886 non-null  int64
 8   windspeed      10886 non-null  float64
 9   casual         10886 non-null  int64
10  registered     10886 non-null  int64
11  count          10886 non-null  int64
12  day            10886 non-null  object
dtypes: datetime64[ns](1), float64(3), int64(8), object(1)
memory usage: 1.1+ MB
```

- The dataframe requires a memory usage of about 1.1+ MB.
- Though the memory usage is small but can we still decrease the memory usage ?

```
[476]: # 1: spring, 2: summer, 3: fall, 4: winter
def season_category(x):
    if x == 1:
        return 'spring'
    elif x == 2:
        return 'summer'
    elif x == 3:
        return 'fall'
    else:
        return 'winter'
df['season'] = df['season'].apply(season_category)
```

1.3 Optimizing Memory Usage of the Dataframe

Updating dtype of season column

```
[477]: print('Memory usage of season column : ', df['season'].memory_usage())
# Since the dtype of season column is object, we can convert the dtype to
# category to save memory
df['season'] = df['season'].astype('category')
print('Updated Memory usage of season column : ', df['season'].memory_usage())
```

Memory usage of season column : 87220
Updated Memory usage of season column : 11222

Updating dtype of season column

```
[478]: print('Max value entry in holiday column : ', df['holiday'].max())
print('Memory usage of holiday column : ', df['holiday'].memory_usage())
# Since the maximum entry in holiday column is 1 and the dtype is int64, we can
↳convert the dtype to category to save memory
df['holiday'] = df['holiday'].astype('category')
print('Updated Memory usage of holiday column : ', df['holiday'].memory_usage())
```

Max value entry in holiday column : 1
Memory usage of holiday column : 87220
Updated Memory usage of holiday column : 11142

Updating dtype of workingday column

```
[479]: print('Max value entry in workingday column : ', df['workingday'].max())
print('Memory usage of workingday column : ', df['workingday'].memory_usage())
# Since the maximum entry in workingday column is 1 and the dtype is int64, we
↳can convert the dtype to category to save memory
df['workingday'] = df['workingday'].astype('category')
print('Updated Memory usage of workingday column : ', df['workingday'].
↳memory_usage())
```

Max value entry in workingday column : 1
Memory usage of workingday column : 87220
Updated Memory usage of workingday column : 11142

Updating dtype of weather column

```
[480]: print('Max value entry in weather column : ', df['weather'].max())
print('Memory usage of weather column : ', df['weather'].memory_usage())
# Since the maximum entry in weather column is 4 and the dtype is int64, we can
↳convert the dtype to category to save memory
df['weather'] = df['weather'].astype('category')
print('Updated Memory usage of weather column : ', df['weather'].memory_usage())
```

Max value entry in weather column : 4
Memory usage of weather column : 87220
Updated Memory usage of weather column : 11222

Updating dtype of temp column

```
[481]: print('Max value entry in temp column : ', df['temp'].max())
print('Memory usage of temp column : ', df['temp'].memory_usage())
# Since the maximum entry in temp column is 41.0 and the dtype is float64, we
↳can convert the dtype to float32 to save memory
df['temp'] = df['temp'].astype('float32')
print('Updated Memory usage of temp column : ', df['temp'].memory_usage())
```

Max value entry in temp column : 41.0
Memory usage of temp column : 87220
Updated Memory usage of temp column : 43676

Updating dtype of atemp column

```
[482]: print('Max value entry in atemp column : ', df['atemp'].max())  
print('Memory usage of atemp column : ', df['atemp'].memory_usage())  
# Since the maximum entry in atemp column is 45.455 and the dtype is float64, we  
# can convert the dtype to float32 to save memory  
df['atemp'] = df['atemp'].astype('float32')  
print('Updated Memory usage of atemp column : ', df['atemp'].memory_usage())
```

Max value entry in atemp column : 45.455
Memory usage of atemp column : 87220
Updated Memory usage of atemp column : 43676

Updating dtype of humidity column

```
[483]: print('Max value entry in humidity column : ', df['humidity'].max())  
print('Memory usage of humidity column : ', df['temp'].memory_usage())  
# Since the maximum entry in humidity column is 100 and the dtype is int64, we  
# can convert the dtype to int8 to save memory  
df['humidity'] = df['humidity'].astype('int8')  
print('Updated Memory usage of humidity column : ', df['humidity'].  
      memory_usage())
```

Max value entry in humidity column : 100
Memory usage of humidity column : 43676
Updated Memory usage of humidity column : 11018

Updating dtype of windspeed column

```
[484]: print('Max value entry in windspeed column : ', df['windspeed'].max())  
print('Memory usage of windspeed column : ', df['windspeed'].memory_usage())  
# Since the maximum entry in windspeed column is 56.9969 and the dtype is  
# float64, we can convert the dtype to float32 to save memory  
df['windspeed'] = df['windspeed'].astype('float32')  
print('Updated Memory usage of windspeed column : ', df['windspeed'].  
      memory_usage())
```

Max value entry in windspeed column : 56.9969
Memory usage of windspeed column : 87220
Updated Memory usage of windspeed column : 43676

Updating dtype of casual column

```
[485]: print('Max value entry in casual column : ', df['casual'].max())  
print('Memory usage of casual column : ', df['casual'].memory_usage())  
# Since the maximum entry in casual column is 367 and the dtype is int64, we  
# can convert the dtype to int16 to save memory  
df['casual'] = df['casual'].astype('int16')
```

```
print('Updated Memory usage of casual column : ', df['casual'].memory_usage())
```

Max value entry in casual column : 367
Memory usage of casual column : 87220
Updated Memory usage of casual column : 21904

Updating dtype of registered column

```
[486]: print('Max value entry in registered column : ', df['registered'].max())
print('Memory usage of registered column : ', df['registered'].memory_usage())
# Since the maximum entry in registered column is 886 and the dtype is int64,
# we can convert the dtype to int16 to save memory
df['registered'] = df['registered'].astype('int16')
print('Updated Memory usage of registered column : ', df['registered'].
      memory_usage())
```

Max value entry in registered column : 886
Memory usage of registered column : 87220
Updated Memory usage of registered column : 21904

Updating dtype of count column

```
[487]: print('Max value entry in count column : ', df['count'].max())
print('Memory usage of count column : ', df['count'].memory_usage())
# Since the maximum entry in count column is 977 and the dtype is int64, we can
# convert the dtype to int16 to save memory
df['count'] = df['count'].astype('int16')
print('Updated Memory usage of count column : ', df['count'].memory_usage())
```

Max value entry in count column : 977
Memory usage of count column : 87220
Updated Memory usage of count column : 21904

```
[488]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 13 columns):
 #   Column        Non-Null Count  Dtype
---  -
 0   datetime      10886 non-null  datetime64[ns]
 1   season        10886 non-null  category
 2   holiday       10886 non-null  category
 3   workingday    10886 non-null  category
 4   weather       10886 non-null  category
 5   temp          10886 non-null  float32
 6   atemp         10886 non-null  float32
 7   humidity      10886 non-null  int8
 8   windspeed     10886 non-null  float32
 9   casual        10886 non-null  int16
10  registered    10886 non-null  int16
```

```

11 count      10886 non-null int16
12 day        10886 non-null object
dtypes: category(4), datetime64[ns](1), float32(3), int16(3), int8(1), object(1)
memory usage: 415.4+ KB

```

Earlier the dataset was using 1.1+ MB of memory but now it has been reduced to 415.2+ KB. Around 63.17 % reduction in the memory usage.

Basic Description of the dataset

```
[489]: df.describe()
```

```
[489]:
```

	datetime	temp	atemp	\
count	10886	10886.000000	10886.000000	
mean	2011-12-27 05:56:22.399411968	20.230862	23.655085	
min	2011-01-01 00:00:00	0.820000	0.760000	
25%	2011-07-02 07:15:00	13.940000	16.665001	
50%	2012-01-01 20:30:00	20.500000	24.240000	
75%	2012-07-01 12:45:00	26.240000	31.059999	
max	2012-12-19 23:00:00	41.000000	45.455002	
std	NaN	7.791590	8.474601	

	humidity	windspeed	casual	registered	count
count	10886.000000	10886.000000	10886.000000	10886.000000	10886.000000
mean	61.886460	12.799396	36.021955	155.552177	191.574132
min	0.000000	0.000000	0.000000	0.000000	1.000000
25%	47.000000	7.001500	4.000000	36.000000	42.000000
50%	62.000000	12.998000	17.000000	118.000000	145.000000
75%	77.000000	16.997900	49.000000	222.000000	284.000000
max	100.000000	56.996899	367.000000	886.000000	977.000000
std	19.245033	8.164537	49.960477	151.039033	181.144454

- These statistics provide insights into the central tendency, spread, and range of the numerical features in the dataset.

```
[490]: np.round(df['season'].value_counts(normalize = True) * 100, 2)
```

```
[490]: season
winter    25.11
fall      25.11
summer    25.11
spring    24.67
Name: proportion, dtype: float64
```

```
[491]: np.round(df['holiday'].value_counts(normalize = True) * 100, 2)
```

```
[491]: holiday
0     97.14
1      2.86
Name: proportion, dtype: float64
```



```
[492]: np.round(df['workingday'].value_counts(normalize = True) * 100, 2)
```

```
[492]: workingday  
1      68.09  
0      31.91  
Name: proportion, dtype: float64
```

```
[493]: np.round(df['weather'].value_counts(normalize = True) * 100, 2)
```

```
[493]: weather  
1      66.07  
2      26.03  
3       7.89  
4       0.01  
Name: proportion, dtype: float64
```

```
[494]: # The below code generates a visually appealing pie chart to showcase the  
       # distribution of seasons in the dataset
```

```
plt.figure(figsize = (6, 6))      # setting the figure size to 6*6
```

```
# setting the title of the plot
```

```
plt.title('Distribution of season', fontdict = {'fontsize' : 18,  
                                                'fontweight' : 600,  
                                                'fontstyle' : 'oblique',  
                                                'fontfamily' : 'serif'})
```

```
df_season = np.round(df['season'].value_counts(normalize=True) * 100, 2).  
    ↳to_frame(name='season')
```

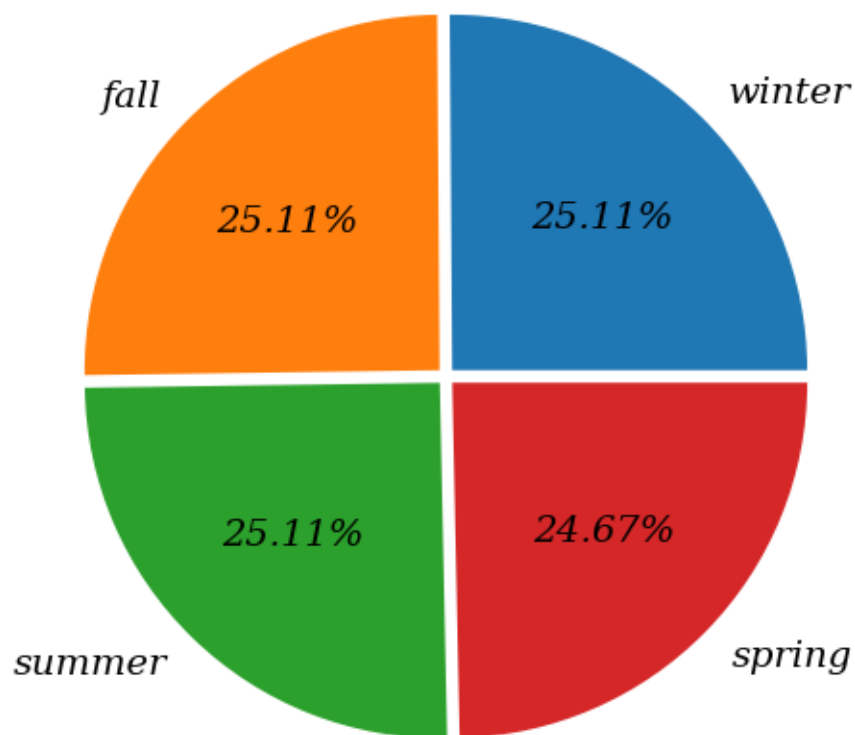
```
# Creating the pie-chart
```

```
plt.pie(x = df_season['season'],  
        explode = [0.025, 0.025, 0.025, 0.025],  
        labels = df_season.index,  
        autopct = '%.2f%%',  
        textprops = {'fontsize' : 14,  
                     'fontstyle' : 'oblique',  
                     'fontfamily' : 'serif',  
                     'fontweight' : 500})
```

```
plt.plot()      # displaying the plot
```

```
[494]: []
```

Distribution of season



```
[495]: # The below code generates a visually appealing pie chart to showcase the
        # distribution of holiday in the dataset

plt.figure(figsize = (6, 6))      # setting the figure size to 6*6

# setting the title of the plot
plt.title('Distribution of holiday', fontdict = {'fontsize' : 18,
                                                'fontweight' : 600,
                                                'fontstyle' : 'oblique',
                                                'fontfamily' : 'serif'})

df_holiday = np.round(df['holiday'].value_counts(normalize = True) * 100, 2).
    ↳to_frame(name='holiday')

# Creating the pie-chart
plt.pie(x = df_holiday['holiday'],
        explode = [0, 0.1],
```

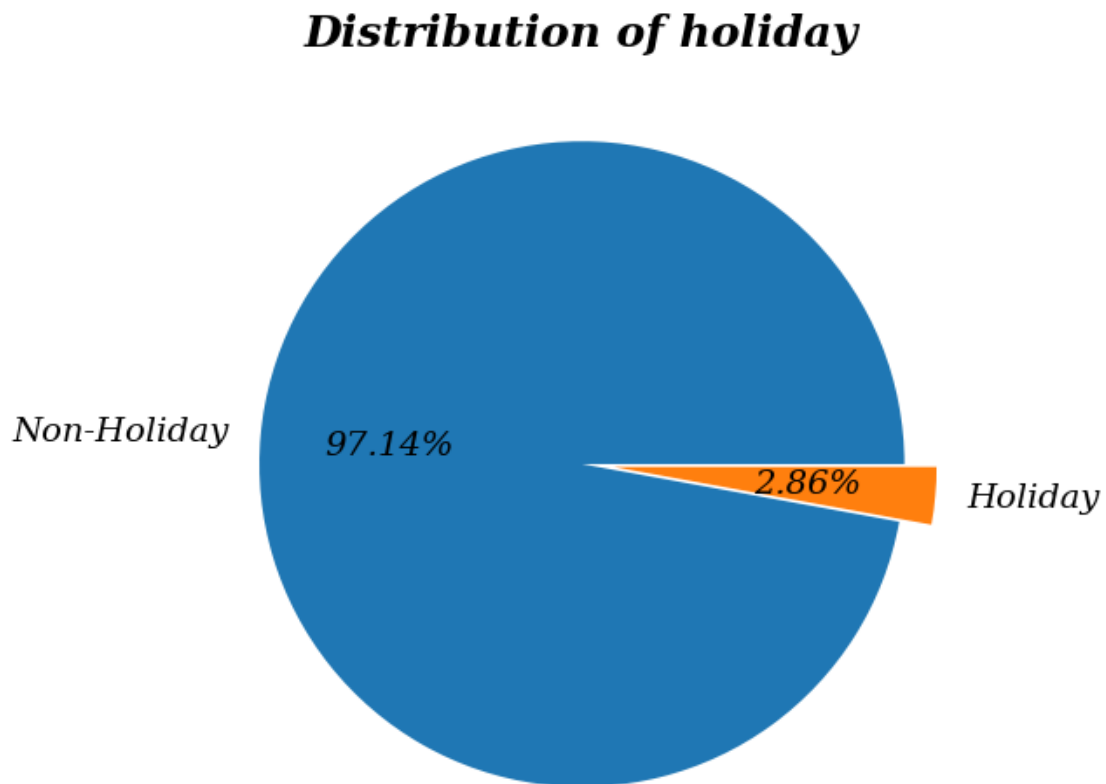
```

labels = ['Non-Holiday', 'Holiday'],
autopct = '%.2f%%',
textprops = {'fontsize' : 14,
              'fontstyle' : 'oblique',
              'fontfamily' : 'serif',
              'fontweight' : 500})

plt.plot()      # displaying the plot

```

[495]: []



[496]: # The below code generates a visually appealing pie chart to showcase the
distribution of workingday in the dataset

```

plt.figure(figsize = (6, 6))    # setting the figure size to 6*6

# setting the title of the plot
plt.title('Distribution of workingday', fontdict = {'fontsize' : 18,
                                                    'fontweight' : 600,

```

```

                                'fontstyle' : 'oblique',
                                'fontfamily' : 'serif'})

df_workingday = np.round(df['workingday'].value_counts(normalize = True) * 100,
↪2).to_frame(name="workingday")

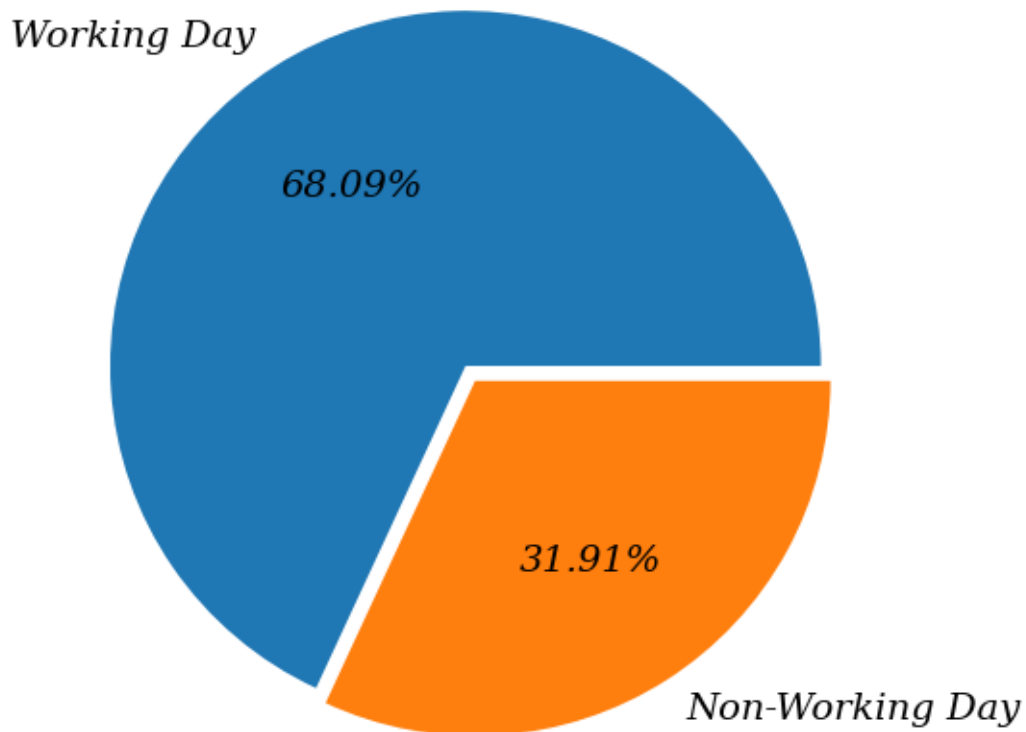
# Creating the pie-chart
plt.pie(x = df_workingday['workingday'],
        explode = [0, 0.05],
        labels = ['Working Day', 'Non-Working Day'],
        autopct = '%.2f%%',
        textprops = {'fontsize' : 14,
                      'fontstyle' : 'oblique',
                      'fontfamily' : 'serif',
                      'fontweight' : 500})

plt.plot()           # displaying the plot

```

[496]: []

Distribution of workingday



```
[497]: # The below code generates a visually appealing pie chart to showcase the
        # distribution of weather in the dataset

plt.figure(figsize = (6, 6))      # setting the figure size to 6*6

# setting the title of the plot
plt.title('Distribution of weather', fontdict = {'fontsize' : 18,
                                                'fontweight' : 600,
                                                'fontstyle' : 'oblique',
                                                'fontfamily' : 'serif'})

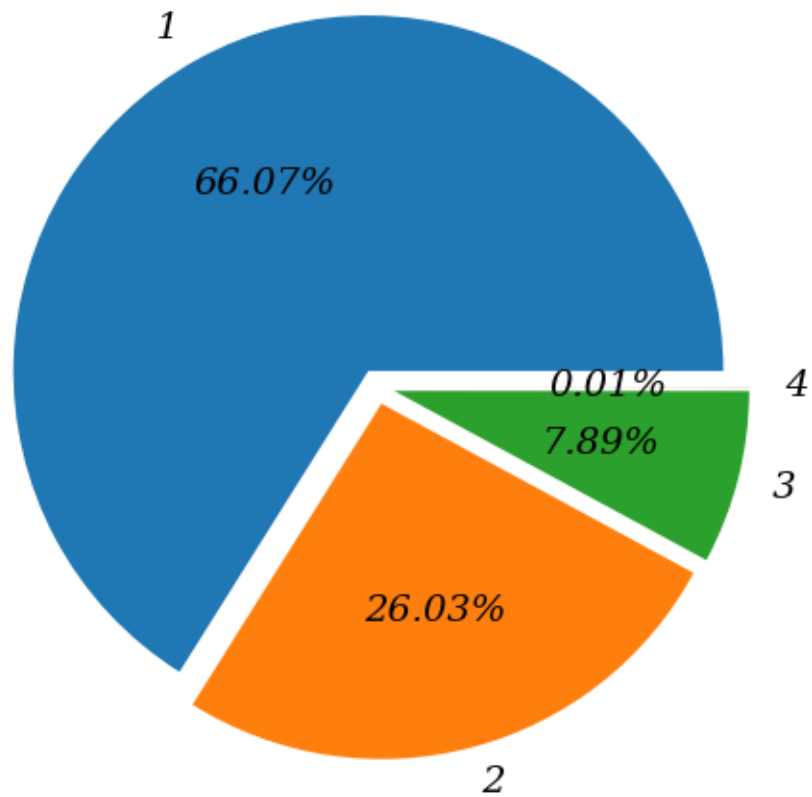
df_weather = np.round(df['weather'].value_counts(normalize = True) * 100, 2).
    ↪to_frame(name="weather")

# Creating the pie-chart
plt.pie(x = df_weather['weather'],
        explode = [0.05, 0.05, 0.05, 0.05],
        labels = df_weather.index,
        autopct = '%.2f%',
        textprops = {'fontsize' : 14,
                    'fontstyle' : 'oblique',
                    'fontfamily' : 'serif',
                    'fontweight' : 500})

plt.plot()      # displaying the plot
```

[497]: []

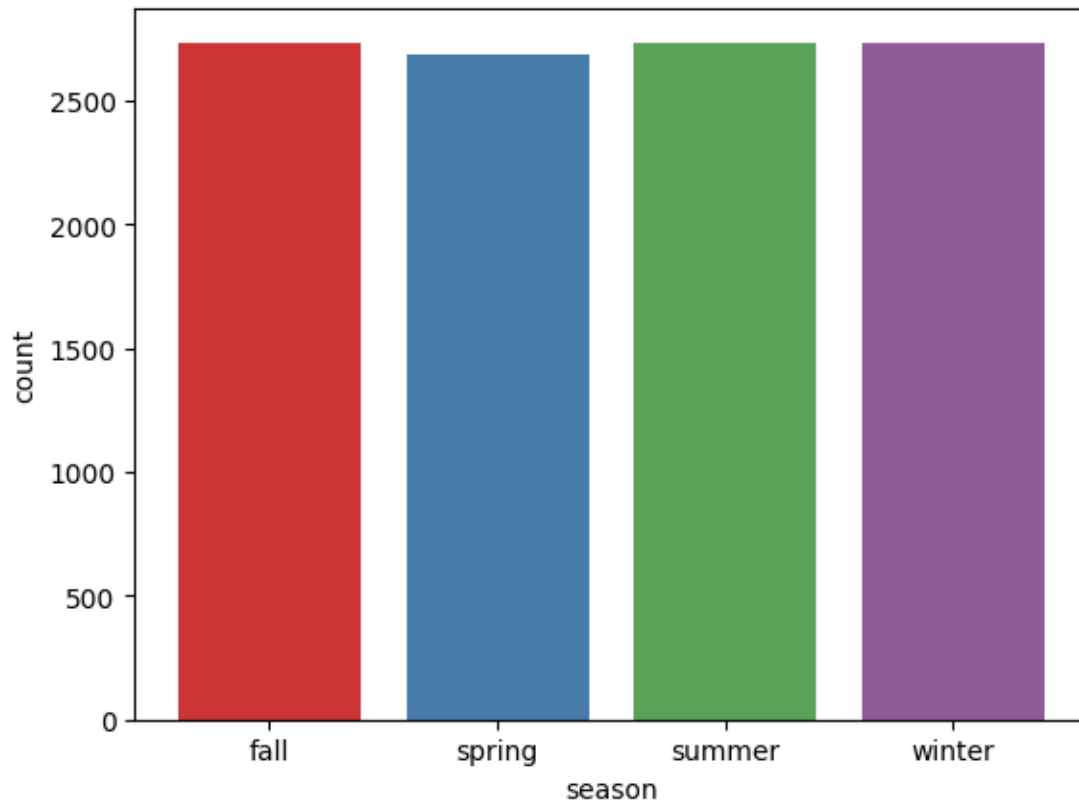
Distribution of weather



1.4 Univariate Analysis

```
[498]: # The below code generates a visually appealing count plot to showcase the  
        # distribution of season in the dataset  
sns.countplot(data = df, x = 'season',palette='Set1')  
plt.plot()
```

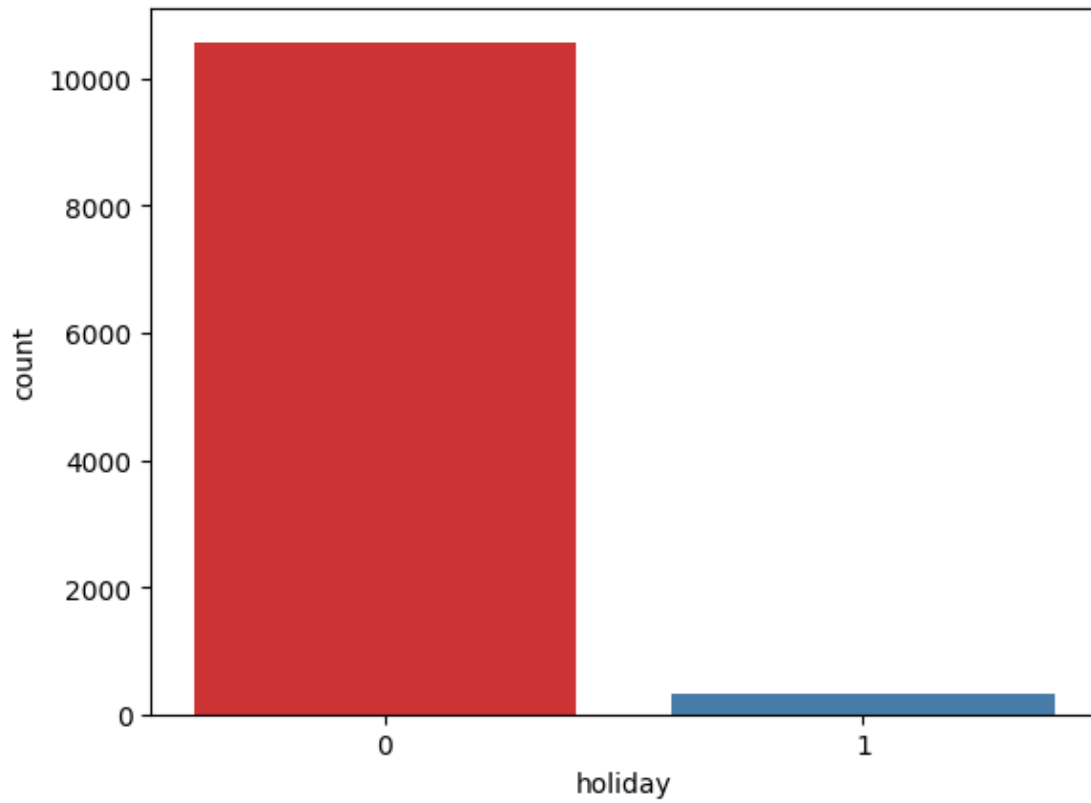
[498]: []



```
[499]: # The below code generates a visually appealing count plot to showcase the  
       # distribution of holiday in the dataset
```

```
sns.countplot(data = df, x = 'holiday',palette='Set1')  
plt.plot()      # displaying the chart
```

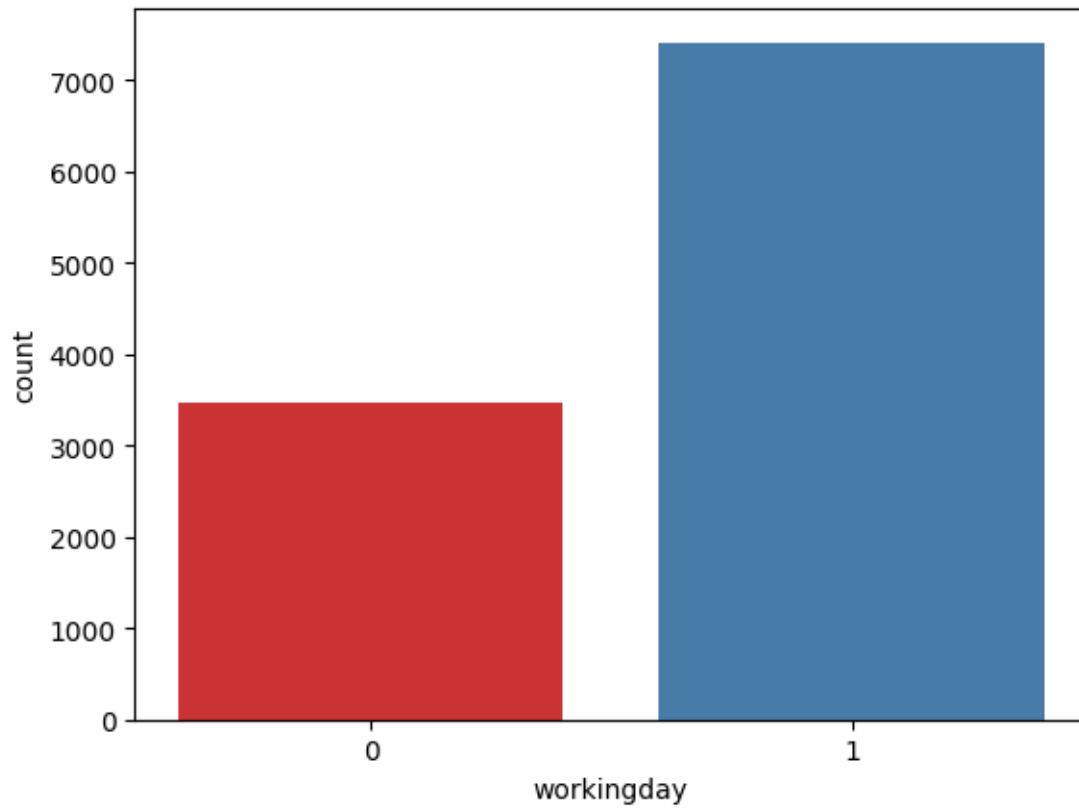
```
[499]: []
```



```
[500]: # The below code generates a visually appealing count plot to showcase the  
        # distribution of workingday in the dataset
```

```
sns.countplot(data = df, x = 'workingday',palette='Set1')  
plt.plot()      # displaying the chart
```

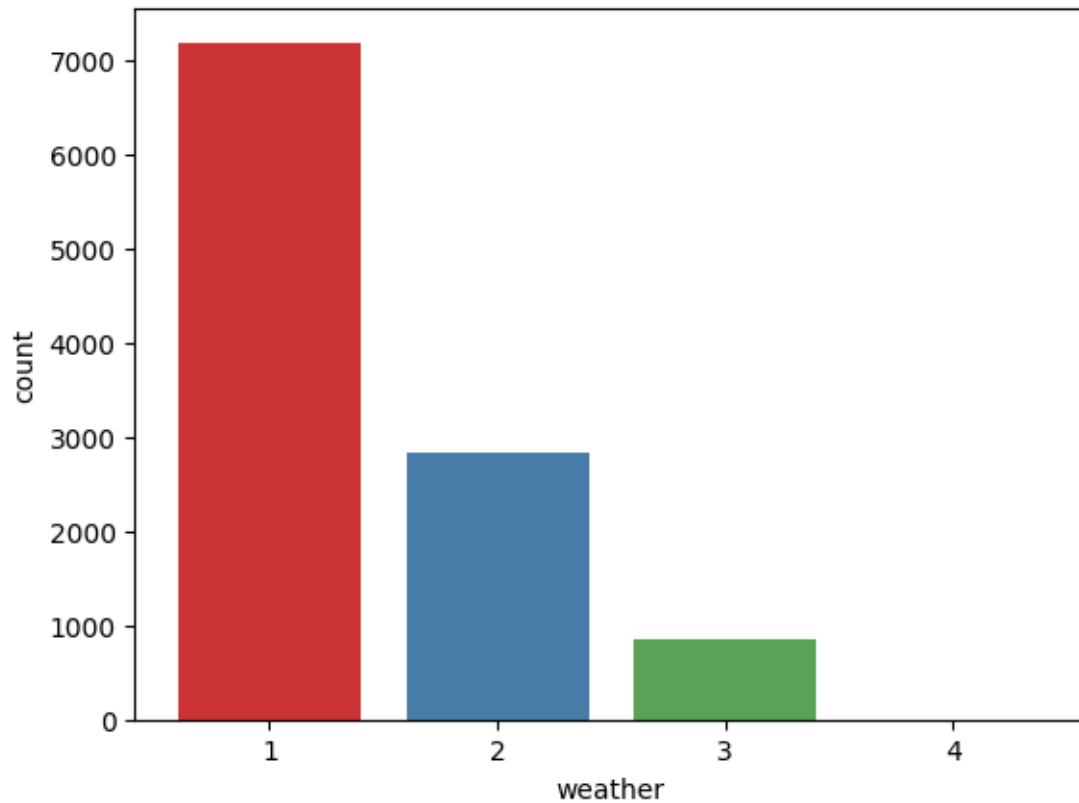
```
[500]: []
```

```
[501]: # The below code generates a visually appealing count plot to showcase the  
       # distribution of weather in the dataset
```

```
sns.countplot(data = df, x = 'weather',palette='Set1')  
plt.plot()      # displaying the chart
```

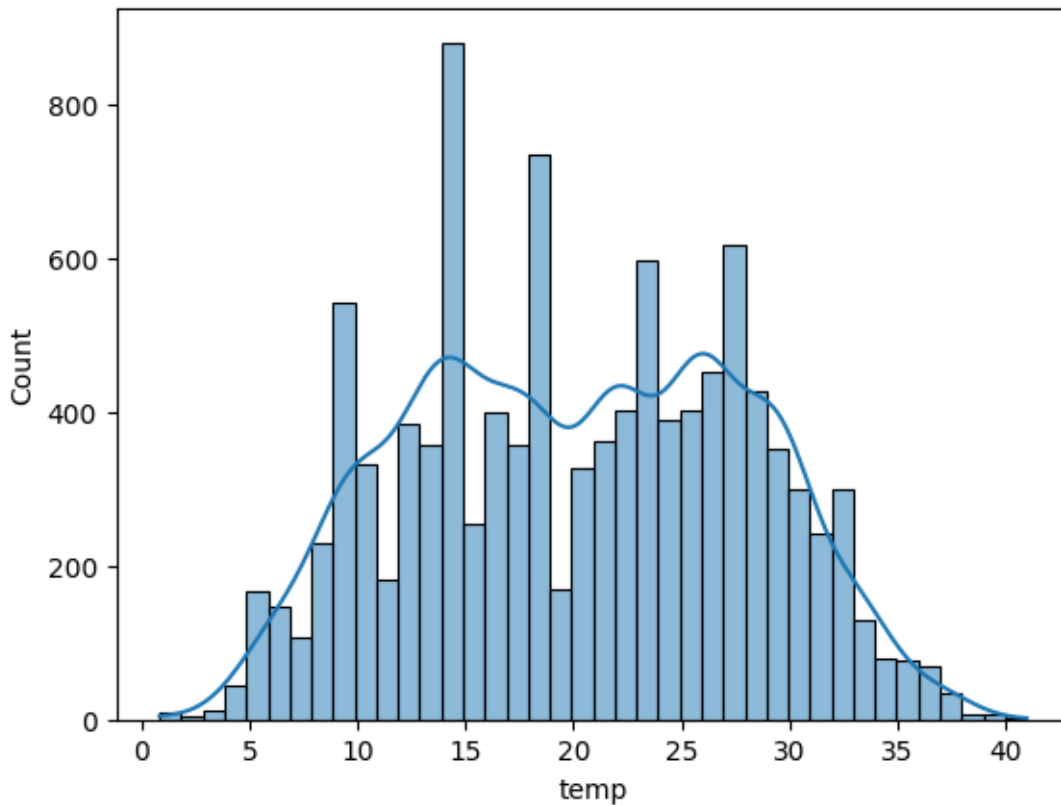
```
[501]: []
```



```
[502]: # The below code generates a histogram plot for the 'temp' feature, showing the
      ↪ distribution of
      # temperature values in the dataset.
      # The addition of the kernel density estimation plot provides
      # a visual representation of the underlying distribution shape, making it
      ↪ easier to analyze the
      # data distribution.

      sns.histplot(data = df, x = 'temp', kde = True, bins = 40)
      plt.plot()
```

[502]: []



```
[503]: temp_mean = np.round(df['temp'].mean(), 2)
temp_std = np.round(df['temp'].std(), 2)
temp_mean, temp_std
```

```
[503]: (np.float32(20.23), np.float32(7.79))
```

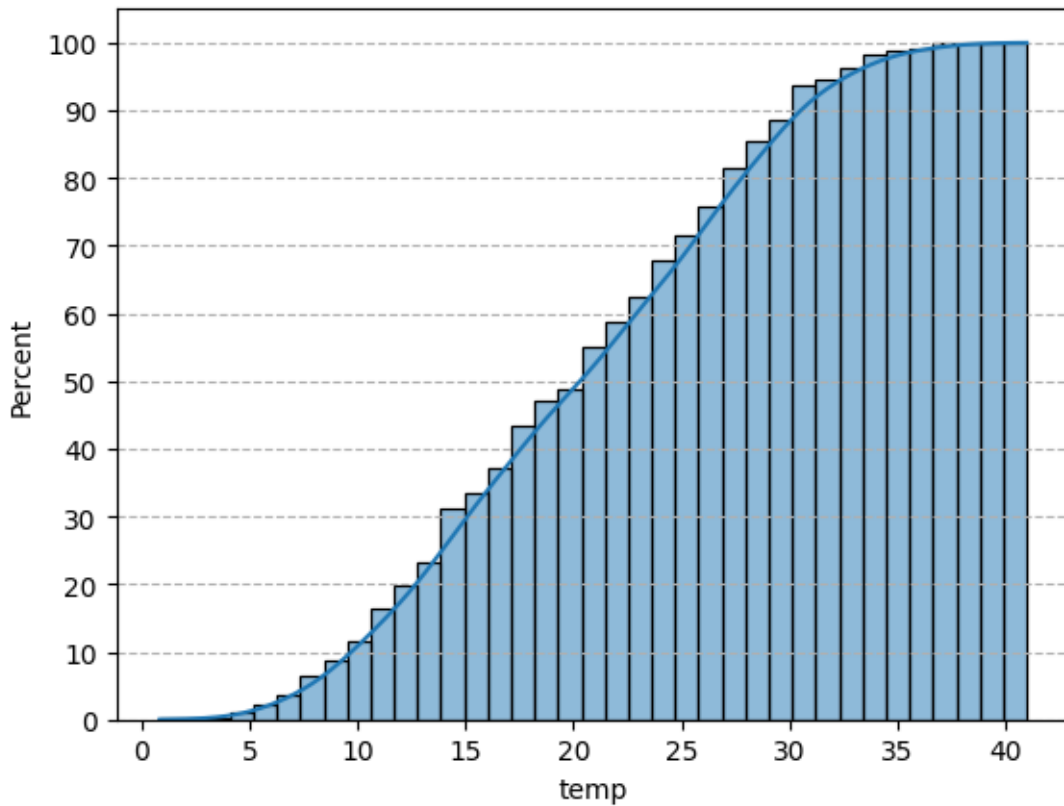
- The mean and the standard deviation of the temp column is 20.23 and 7.79 degree celcius respectively.

```
[504]: # The below code generates a histogram plot for the 'temp' feature, showing the
      ↪ cumulative
      # distribution of temperature values in the dataset.
# The addition of the kernel density estimation plot provides
      # a visual representation of the underlying distribution shape, making it
      ↪ easier to analyze the
      # data distribution.

sns.histplot(data = df, x = 'temp', kde = True, cumulative = True, stat =
      ↪ 'percent')
plt.grid(axis = 'y', linestyle = '--')
plt.yticks(np.arange(0, 101, 10))
```

```
plt.plot()          # displaying the chart
```

[504]: []

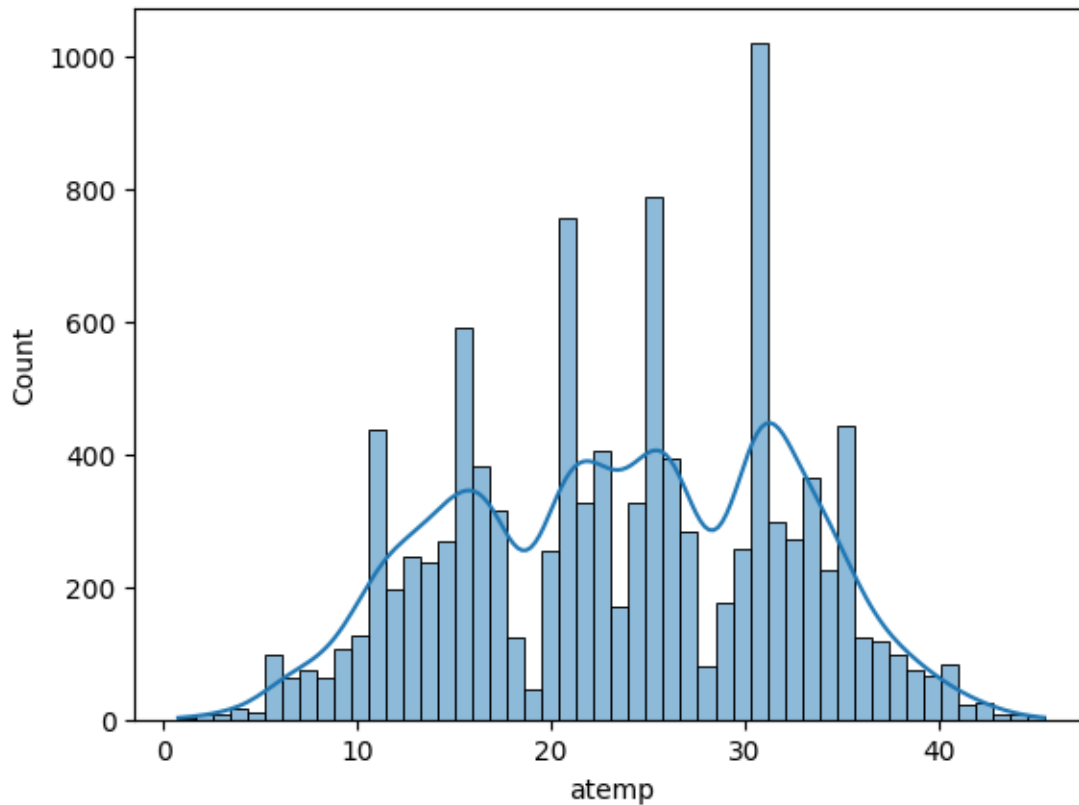


- More than 80 % of the time, the temperature is less than 28 degrees celcius.

```
[505]: # The below code generates a histogram plot for the 'atemp' feature, showing
        ↳ the distribution of
          # feeling temperature values in the dataset.
        # The addition of the kernel density estimation plot provides
          # a visual representation of the underlying distribution shape, making it
        ↳ easier to analyze the
          # data distribution.

sns.histplot(data = df, x = 'atemp', kde = True, bins = 50)
plt.plot()          # displaying the chart
```

[505]: []



```
[506]: temp_mean = np.round(df['atemp'].mean(), 2)
temp_std = np.round(df['atemp'].std(), 2)
temp_mean, temp_std
```

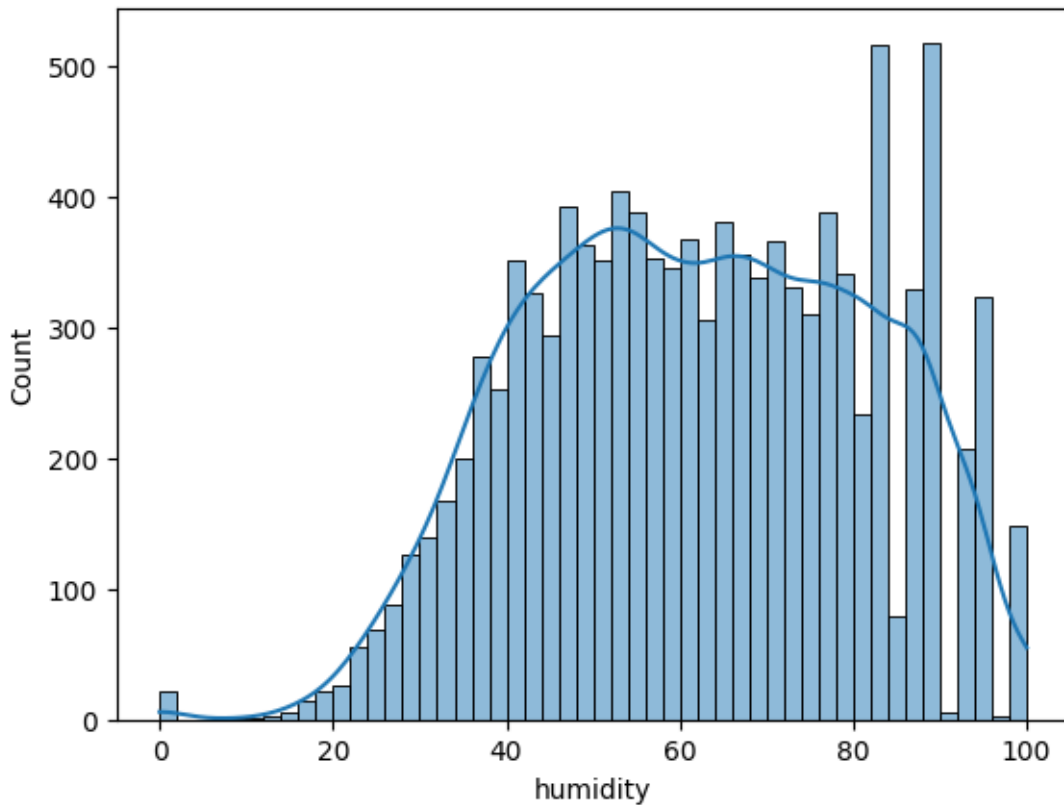
```
[506]: (np.float32(23.66), np.float32(8.47))
```

- The mean and the standard deviation of the atemp column is 23.66 and 8.47 degree celcius respectively.

```
[507]: # The below code generates a histogram plot for the 'humidity' feature, showing
# the distribution of
# humidity values in the dataset.
# The addition of the kernel density estimation plot provides
# a visual representation of the underlying distribution shape, making it
# easier to analyze the
# data distribution.

sns.histplot(data = df, x = 'humidity', kde = True, bins = 50)
plt.plot() # displaying the chart
```

```
[507]: []
```



```
[508]: humidity_mean = np.round(df['humidity'].mean(), 2)
humidity_std = np.round(df['humidity'].std(), 2)
humidity_mean, humidity_std
```

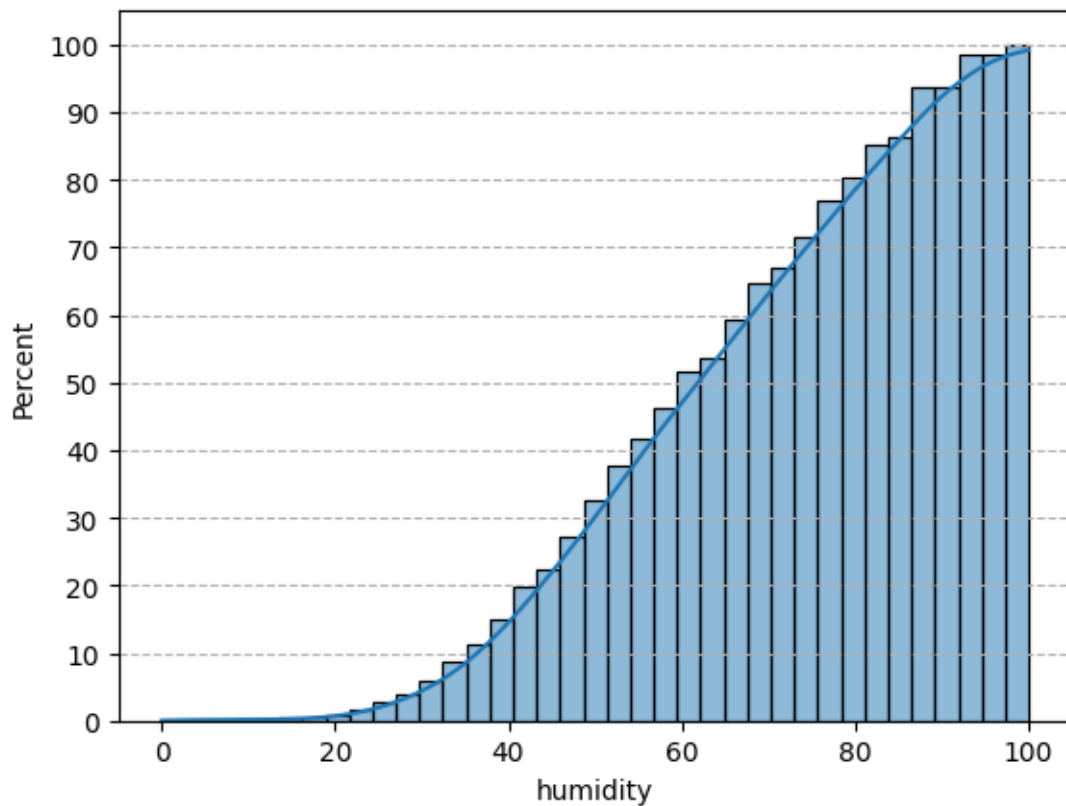
```
[508]: (np.float64(61.89), np.float64(19.25))
```

- The mean and the standard deviation of the humidity column is 61.89 and 19.25 respectively.

```
[509]: # The below code generates a histogram plot for the 'humidity' feature, showing
# the cumulative
# distribution of humidity values in the dataset.
# The addition of the kernel density estimation plot provides
# a visual representation of the underlying distribution shape, making it
# easier to analyze the
# data distribution.

sns.histplot(data = df, x = 'humidity', kde = True, cumulative = True, stat =
# 'percent')
plt.grid(axis = 'y', linestyle = '--') # setting the gridlines along y axis
plt.yticks(np.arange(0, 101, 10))
plt.plot() # displaying the chart
```

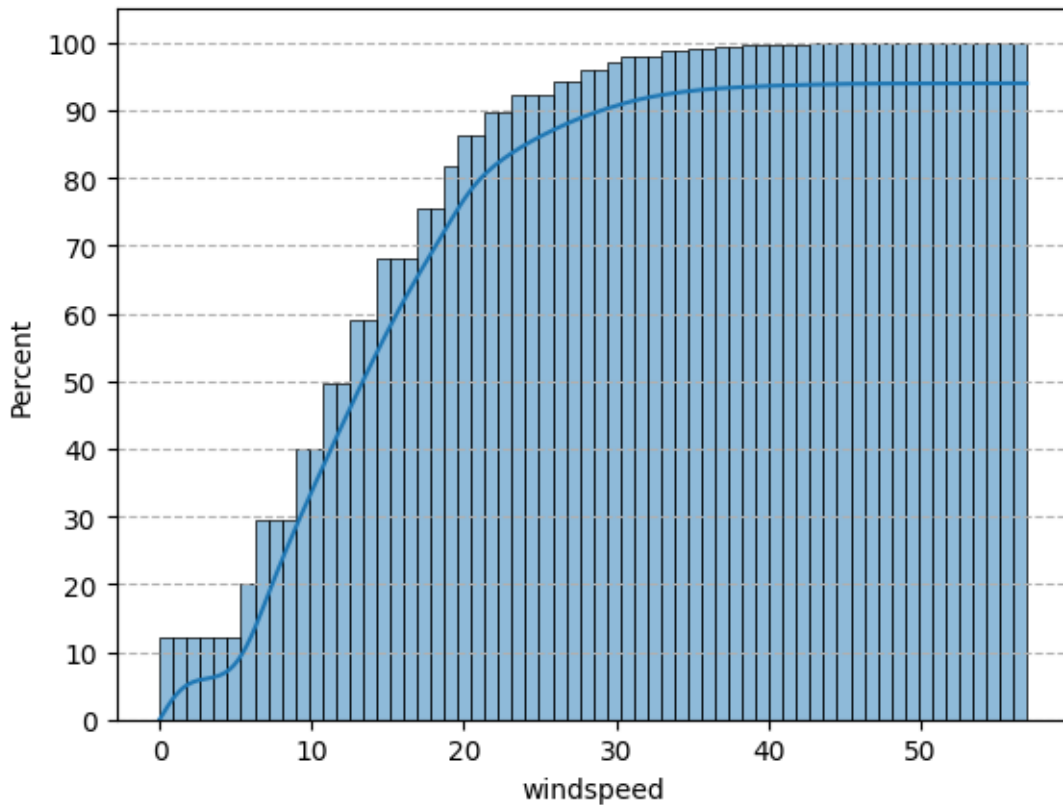
[509]: []



- More than 80 % of the time, the humidity value is greater than 40. Thus for most of the time, humidity level varies from optimum to too moist.

```
[510]: sns.histplot(data = df, x = 'windspeed', kde = True, cumulative = True, stat = 'percent')
plt.grid(axis = 'y', linestyle = '--')
plt.yticks(np.arange(0, 101, 10))
plt.plot()      # displaying the chart
```

[510]: []



- More than 85 % of the total windspeed data has a value of less than 20.

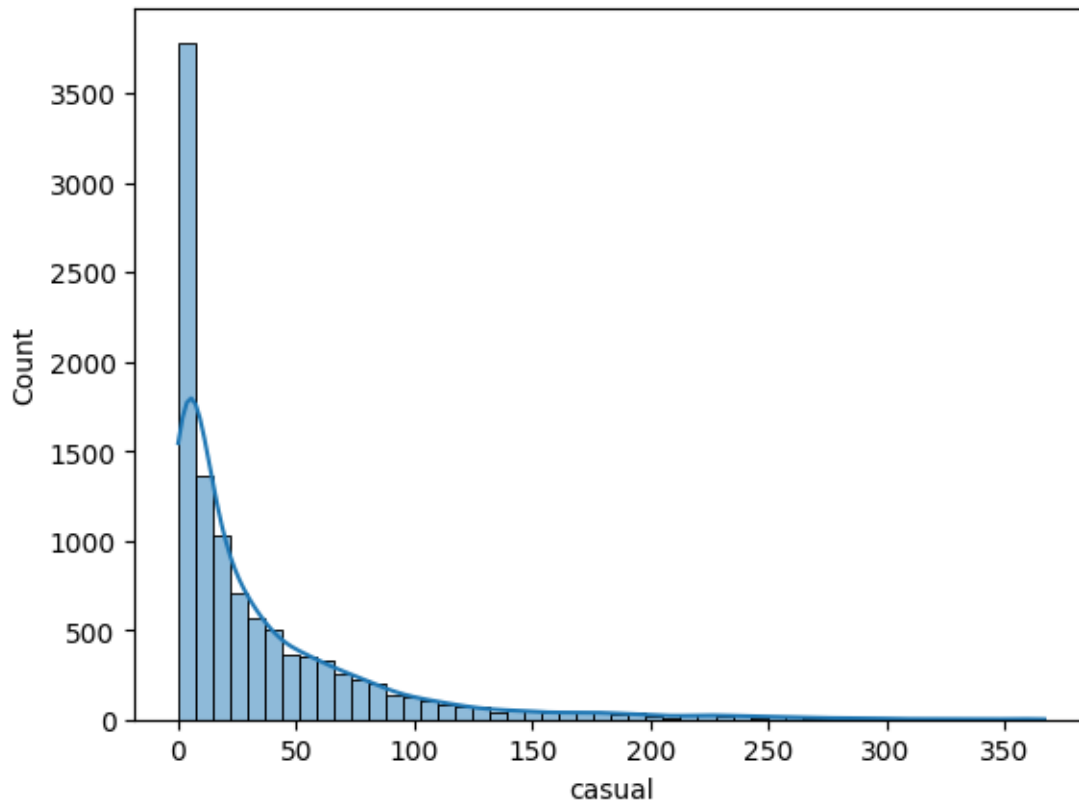
```
[511]: len(df[df['windspeed'] < 20]) / len(df)
```

```
[511]: 0.8626676465184641
```

```
[512]: # The below code generates a histogram plot for the 'casual' feature, showing
        ↳ the distribution of
        # casual users' values in the dataset.
        # The addition of the kernel density estimation plot provides
        # a visual representation of the underlying distribution shape, making it
        ↳ easier to analyze the
        # data distribution.

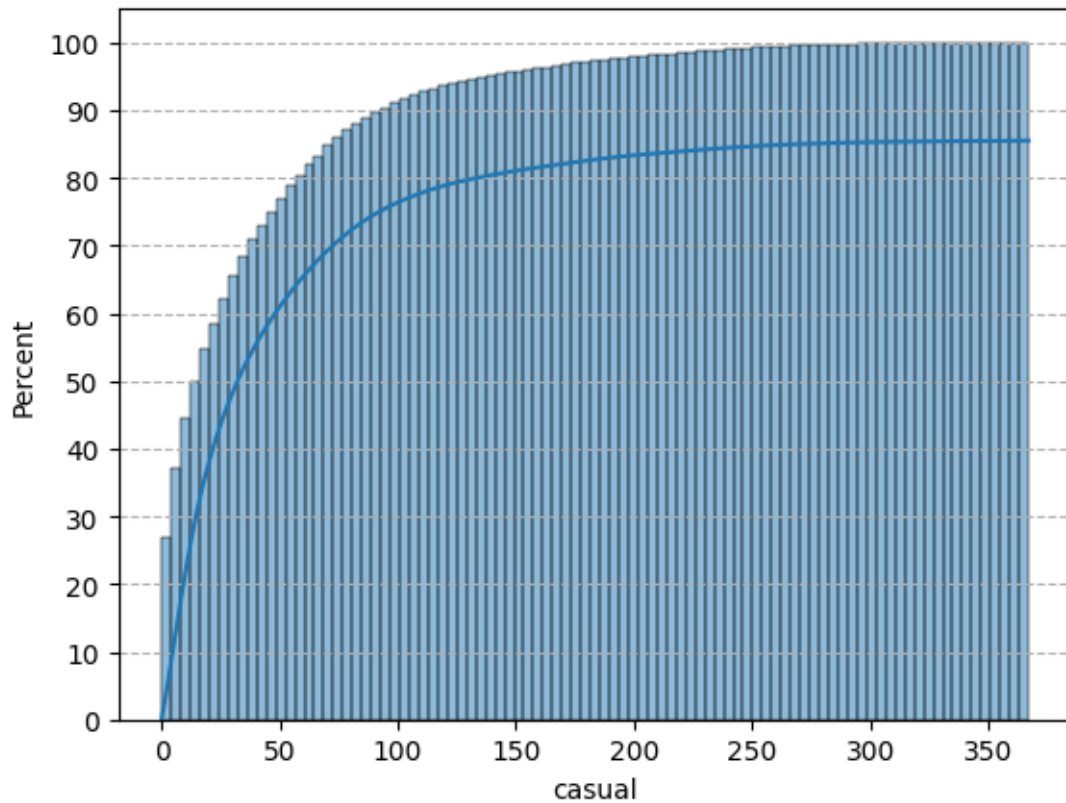
        sns.histplot(data = df, x = 'casual', kde = True, bins = 50)
        plt.plot()      # displaying the chart
```

```
[512]: []
```

```
[513]: sns.histplot(data = df, x = 'casual', kde = True, cumulative = True, stat = 'percent')
plt.grid(axis = 'y', linestyle = '--')
plt.yticks(np.arange(0, 101, 10))
plt.plot()      # displaying the chart
```

[513]: []

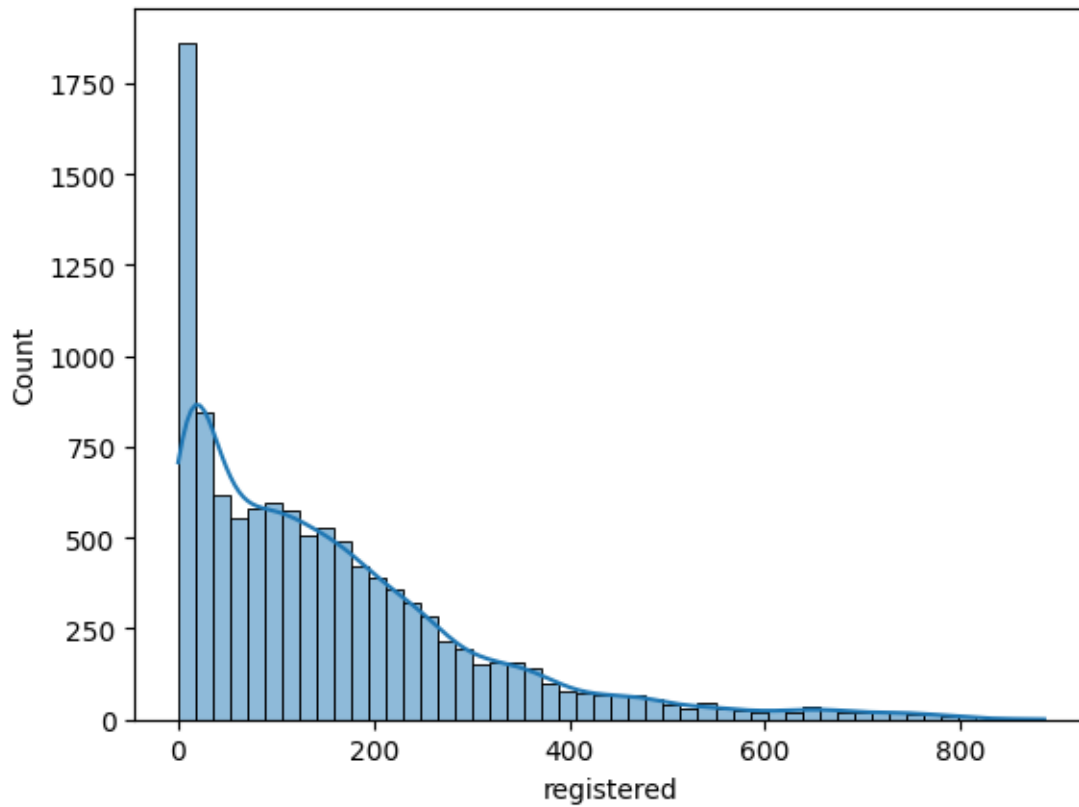


- More than 80 % of the time, the count of casual users is less than 60.

```
[514]: # The below code generates a histogram plot for the 'registered' feature,
        # showing the distribution of
        # registered users' values in the dataset.
        # The addition of the kernel density estimation plot provides
        # a visual representation of the underlying distribution shape, making it
        # easier to analyze the
        # data distribution.

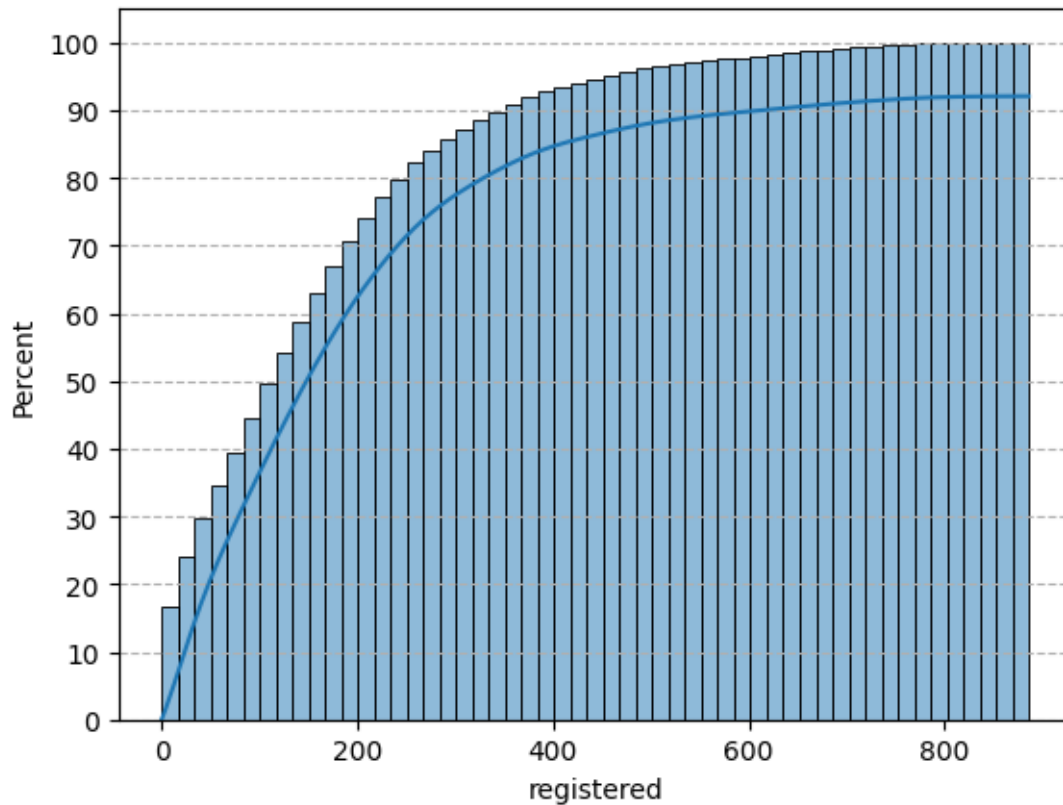
sns.histplot(data = df, x = 'registered', kde = True, bins = 50)
plt.plot()      # displaying the chart
```

[514]: []



```
[515]: sns.histplot(data = df, x = 'registered', kde = True, cumulative = True, stat = 'percent')
plt.grid(axis = 'y', linestyle = '--')
plt.yticks(np.arange(0, 101, 10))
plt.plot()      # displaying the chart
```

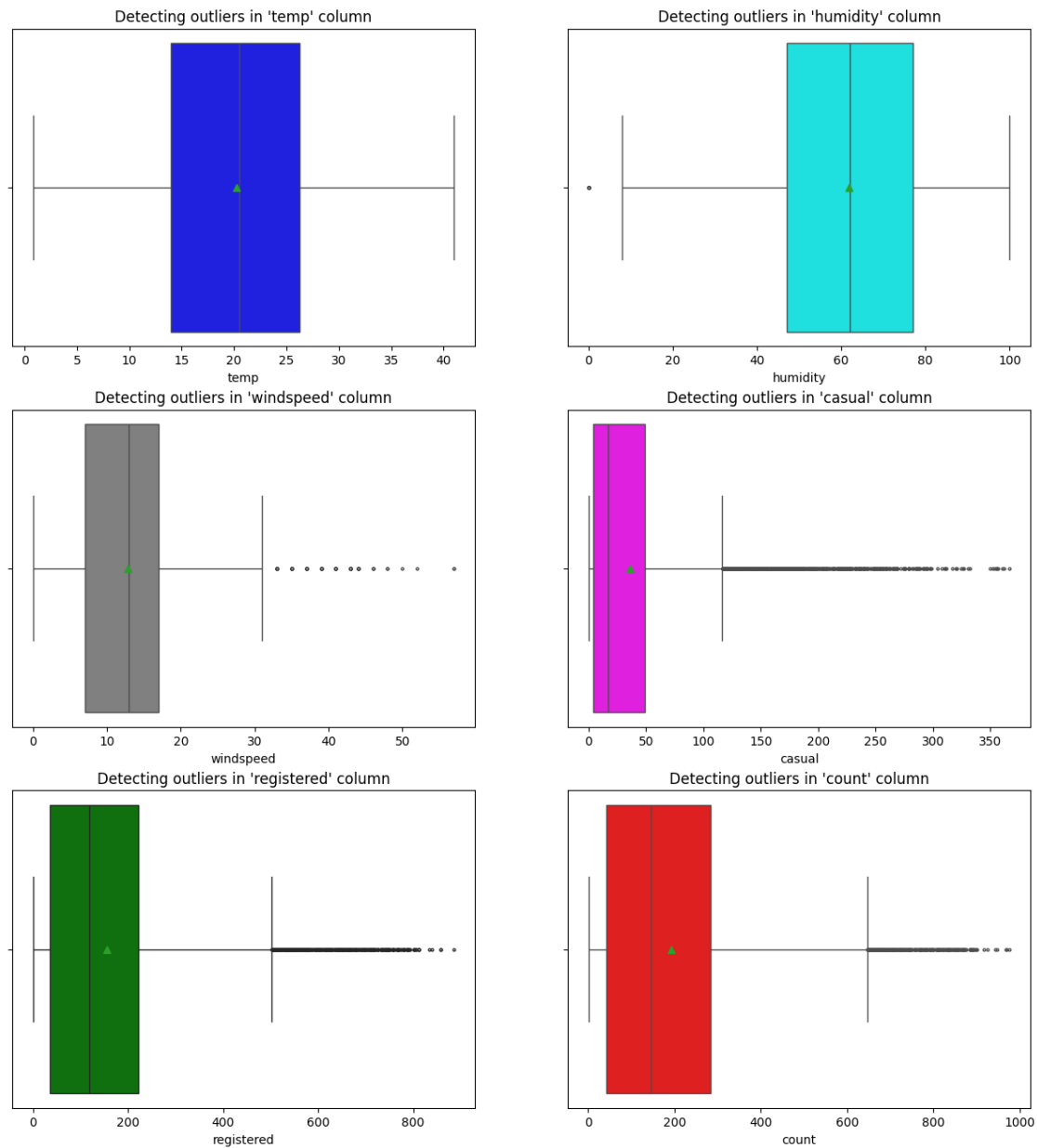
[515]: []



- More than 85 % of the time, the count of registered users is less than 300.

1.5 Outliers Detection

```
[516]: columns = ['temp', 'humidity', 'windspeed', 'casual', 'registered', 'count']
colors = np.random.permutation(['red', 'blue', 'green', 'magenta', 'cyan', 'gray'])
count = 1
plt.figure(figsize = (15, 16))
for i in columns:
    plt.subplot(3, 2, count)
    plt.title(f"Detecting outliers in '{i}' column")
    sns.boxplot(data = df, x = df[i], color = colors[count - 1], showmeans = True, fliersize = 2)
    plt.plot()
    count += 1
```

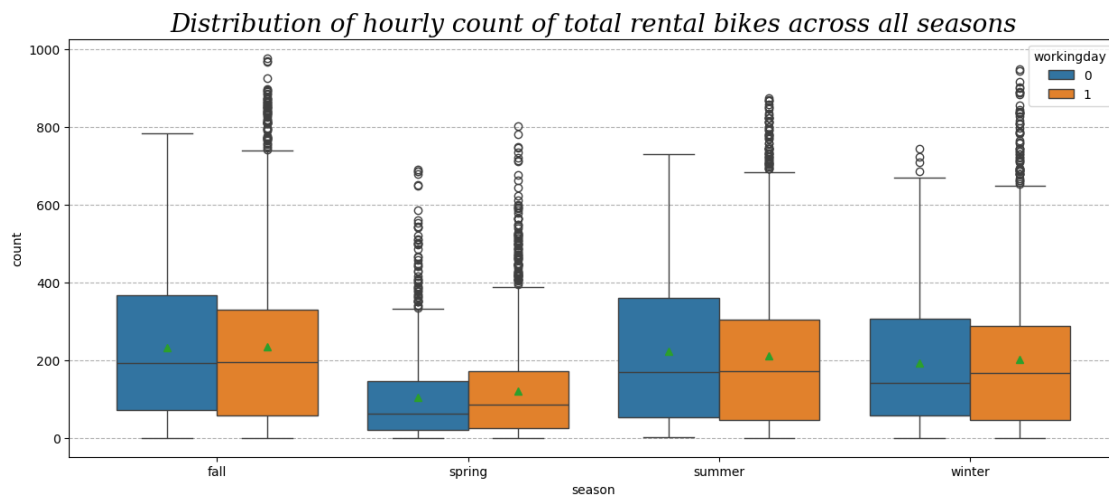


- There is no outlier in the temp column.
- There are few outliers present in humidity column.
- There are many outliers present in each of the columns : windspeed, casual, registered, count.

1.6 Bivariate Analysis

```
[517]: plt.figure(figsize = (15, 6))
plt.title('Distribution of hourly count of total rental bikes across all_
↳seasons',
        fontdict = {'size' : 20,
                    'style' : 'oblique',
                    'family' : 'serif'})
sns.boxplot(data = df, x = 'season', y = 'count', hue = 'workingday', showmeans_
↳= True)
plt.grid(axis = 'y', linestyle = '--')
plt.plot()
```

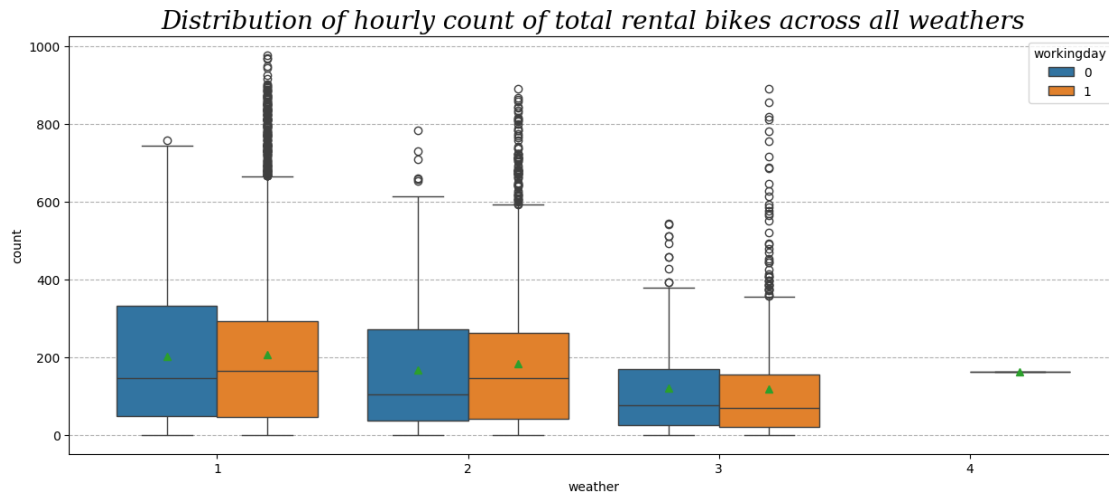
[517]: []



- The hourly count of total rental bikes is higher in the fall season, followed by the summer and winter seasons. It is generally low in the spring season.

```
[518]: plt.figure(figsize = (15, 6))
plt.title('Distribution of hourly count of total rental bikes across all_
↳weathers',
        fontdict = {'size' : 20,
                    'style' : 'oblique',
                    'family' : 'serif'})
sns.boxplot(data = df, x = 'weather', y = 'count', hue = 'workingday',_
↳showmeans = True)
plt.grid(axis = 'y', linestyle = '--')
plt.plot()
```

[518]: []



- The hourly count of total rental bikes is higher in the clear and cloudy weather, followed by the misty weather and rainy weather. There are very few records for extreme weather conditions.

1.7 1) Is there any effect of Working Day on the number of electric cycles rented ?

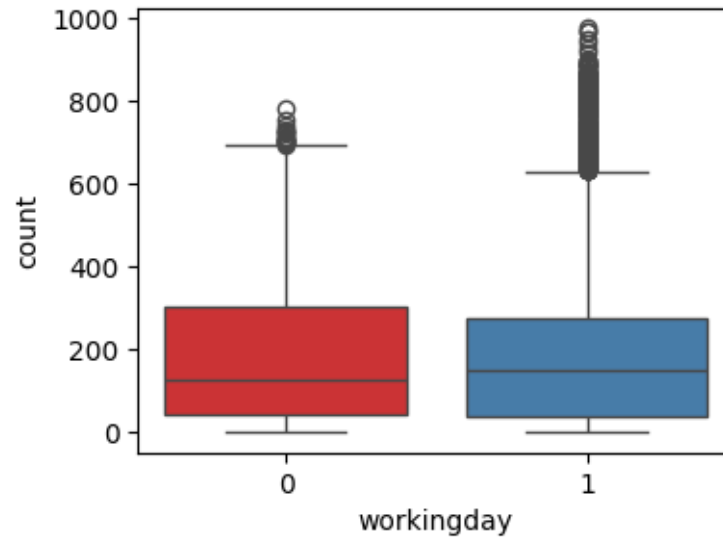
```
[519]: df.groupby(by = 'workingday')['count'].describe()
```

```
[519]:
```

	count	mean	std	min	25%	50%	75%	max
workingday								
0	3474.0	188.506621	173.724015	1.0	44.0	128.0	304.0	783.0
1	7412.0	193.011873	184.513659	1.0	41.0	151.0	277.0	977.0

```
[520]: plt.figure(figsize = (4, 3))
sns.boxplot(data = df, x = 'workingday', y = 'count',palette='Set1')
plt.plot()
```

```
[520]: []
```



STEP-1 : Set up Null Hypothesis

- Null Hypothesis (H_0) - Working Day does not have any effect on the number of electric cycles rented.
- Alternate Hypothesis (H_A) - Working Day has some effect on the number of electric cycles rented

STEP-2 : Checking for basic assumptions for the hypothesis

- Distribution check using QQ Plot
- homogeneity of Variances using Levene's test

STEP-3: Define Test statistics; Distribution of T under H_0 .

- If the assumptions of T Test are met then we can proceed performing T Test for independent samples else we will perform the non parametric test equivalent to T Test for independent sample i.e., Mann-Whitney U rank test for two independent samples.

STEP-4: Compute the p-value and fix value of alpha.

- We set our alpha to be 0.05

STEP-5: Compare p-value and alpha.

- Based on p-value, we will accept or reject H_0 .

$p\text{-val} > \alpha$: Accept H_0

$p\text{-val} < \alpha$: Reject H_0

Visual Tests to know if the samples follow normal distribution

```
[521]: plt.figure(figsize = (15, 5))
plt.subplot(1, 2, 1)
```

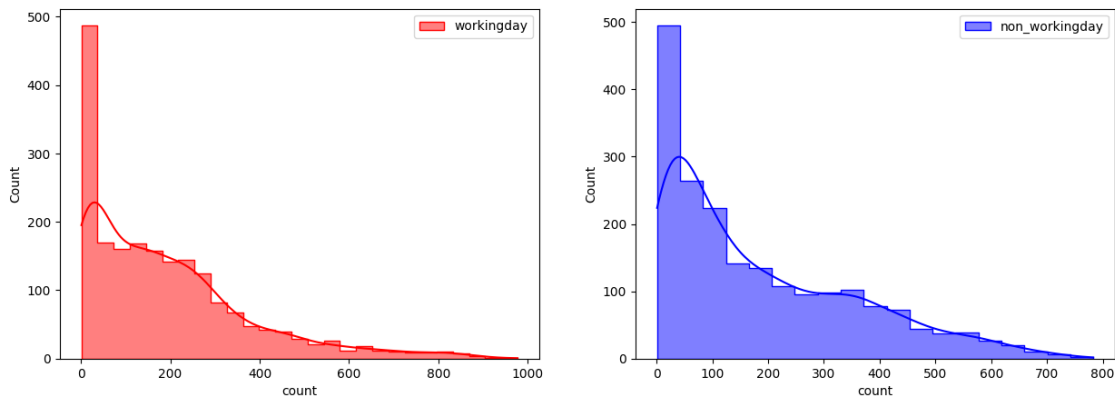


```

sns.histplot(df.loc[df['workingday'] == 1, 'count'].sample(2000),
             element = 'step', color = 'red', kde = True, label = 'workingday')
plt.legend()
plt.subplot(1, 2, 2)
sns.histplot(df.loc[df['workingday'] == 0, 'count'].sample(2000),
             element = 'step', color = 'blue', kde = True, label = 'non_workingday')
plt.legend()
plt.plot()

```

[521]: []



- It can be inferred from the above plot that the distributions do not follow normal distribution.

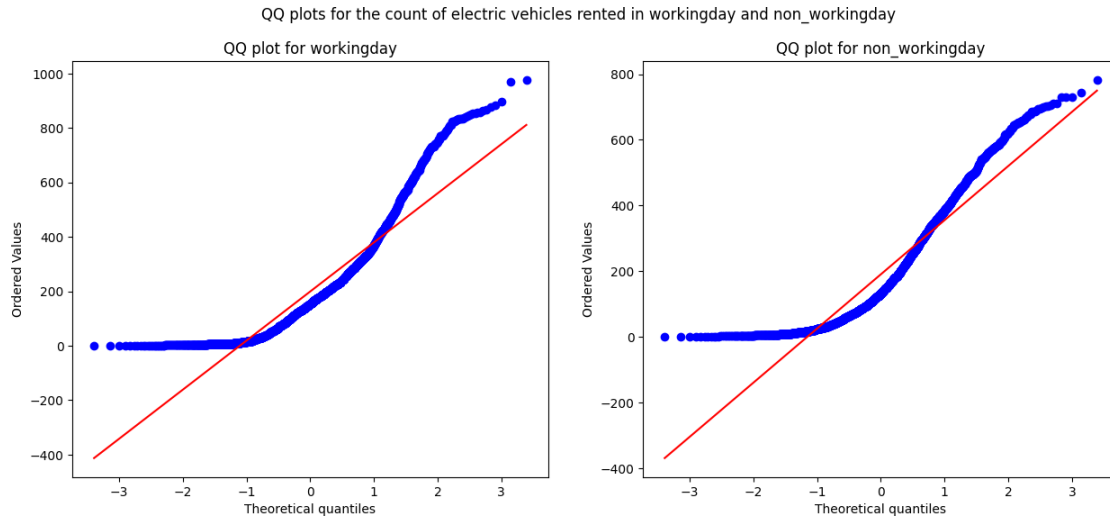
Distribution check using QQ Plot

```

[522]: plt.figure(figsize = (15, 6))
plt.subplot(1, 2, 1)
plt.suptitle('QQ plots for the count of electric vehicles rented in workingday_
and non_workingday')
spy.probplot(df.loc[df['workingday'] == 1, 'count'].sample(2000), plot = plt,
dist = 'norm')
plt.title('QQ plot for workingday')
plt.subplot(1, 2, 2)
spy.probplot(df.loc[df['workingday'] == 0, 'count'].sample(2000), plot = plt,
dist = 'norm')
plt.title('QQ plot for non_workingday')
plt.plot()

```

[522]: []



- It can be inferred from the above plot that the distributions do not follow normal distribution.

It can be seen from the above plots that the samples do not come from normal distribution.

- Applying Shapiro-Wilk test for normality
- H_0 - The sample follows normal distribution
- H_1 - The sample does not follow normal distribution
- $\alpha = 0.05$
- Test Statistics : Shapiro-Wilk test for normality

```
[523]: test_stat, p_value = spy.shapiro(df.loc[df['workingday'] == 1, 'count'].
        ↪sample(2000))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 9.753568384584938e-39

The sample does not follow normal distribution

```
[524]: test_stat, p_value = spy.shapiro(df.loc[df['workingday'] == 0, 'count'].
        ↪sample(2000))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 1.9665619971363844e-35
The sample does not follow normal distribution

Transforming the data using boxcox transformation and checking if the transformed data follows normal distribution.

```
[525]: import warnings

# Assuming you have loaded your data into a DataFrame 'df'
transformed_workingday = spy.boxcox(df.loc[df['workingday'] == 1, 'count'])[0]

# Ignore the UserWarning
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    test_stat, p_value = spy.shapiro(transformed_workingday)

print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows a normal distribution')
```

p-value 1.606448460668464e-33
The sample does not follow normal distribution

```
[526]: transformed_non_workingday = spy.boxcox(df.loc[df['workingday'] == 1,
↳ 'count'])[0]

# Ignore the UserWarning
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    test_stat, p_value = spy.shapiro(transformed_workingday)

print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows a normal distribution')
```

p-value 1.606448460668464e-33
The sample does not follow normal distribution

- Even after applying the boxcox transformation on each of the “workingday” and “non_workingday” data, the samples do not follow normal distribution.
- Homogeneity of Variances using Lavene’s test

```
[527]: # Null Hypothesis(H0) - Homogenous Variance

# Alternate Hypothesis(HA) - Non Homogenous Variance
```

```
test_stat, p_value = spy.levene(df.loc[df['workingday'] == 1, 'count'].
    ↪sample(2000),
                                df.loc[df['workingday'] == 0, 'count'].
    ↪sample(2000))
print('p-value', p_value)
if p_value < 0.05:
    print('The samples do not have Homogenous Variance')
else:
    print('The samples have Homogenous Variance ')
```

p-value 0.8443992321870682
The samples have Homogenous Variance

Since the samples are not normally distributed, T-Test cannot be applied here, we can perform its non parametric equivalent test i.e., Mann-Whitney U rank test for two independent samples.

```
[528]: test_stat, p_value = spy.mannwhitneyu(df.loc[df['workingday'] == 1, 'count'],
                                             df.loc[df['workingday'] == 0, 'count'])
print('P-value :', p_value)
if p_value < 0.05:
    print('Mean no.of electric cycles rented is not same for working and ↵
    ↪non-working days')
else:
    print('Mean no.of electric cycles rented is same for working and ↵
    ↪non-working days')
```

P-value : 0.9679139953914079
Mean no.of electric cycles rented is same for working and non-working days

Therefore, the mean hourly count of the total rental bikes is statistically same for both working and non- working days .

2) Is there any effect of holidays on the number of electric cycles rented ?

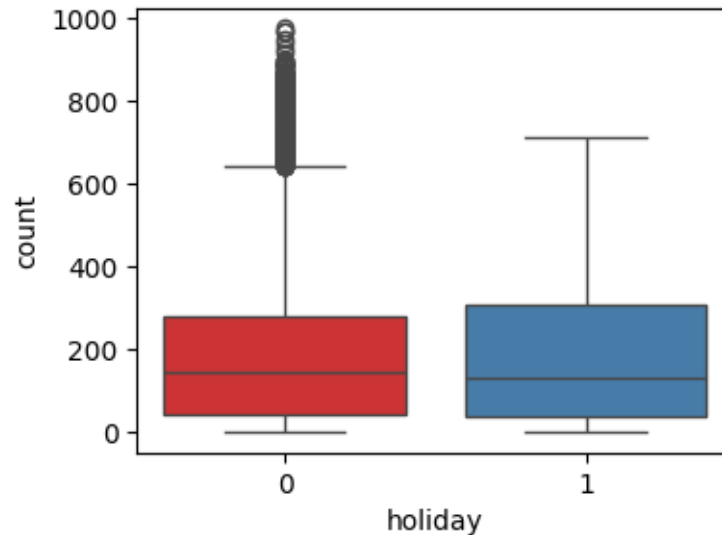
```
[529]: df.groupby(by = 'holiday')['count'].describe()
```

```
[529]:
```

	count	mean	std	min	25%	50%	75%	max
holiday								
0	10575.0	191.741655	181.513131	1.0	43.0	145.0	283.0	977.0
1	311.0	185.877814	168.300531	1.0	38.5	133.0	308.0	712.0

```
[530]: plt.figure(figsize = (4, 3))
sns.boxplot(data = df, x = 'holiday', y = 'count', palette='Set1')
plt.plot()
```

```
[530]: []
```



STEP-1 : Set up Null Hypothesis

- Null Hypothesis (H_0) - Holidays have no effect on the number of electric vehicles rented
- Alternate Hypothesis (H_A) - Holidays has some effect on the number of electric vehicles rented

STEP-2 : Checking for basic assumptions for the hypothesis

- Distribution check using QQ Plot
- Homogeneity of Variances using Levene's test

STEP-3: Define Test statistics; Distribution of T under H_0 .

- If the assumptions of T Test are met then we can proceed performing T Test for independent samples else we will perform the non parametric test equivalent to T Test for independent sample i.e., Mann-Whitney U rank test for two independent samples.

STEP-4: Compute the p-value and fix value of alpha.

- We set our alpha to be 0.05

STEP-5: Compare p-value and alpha.

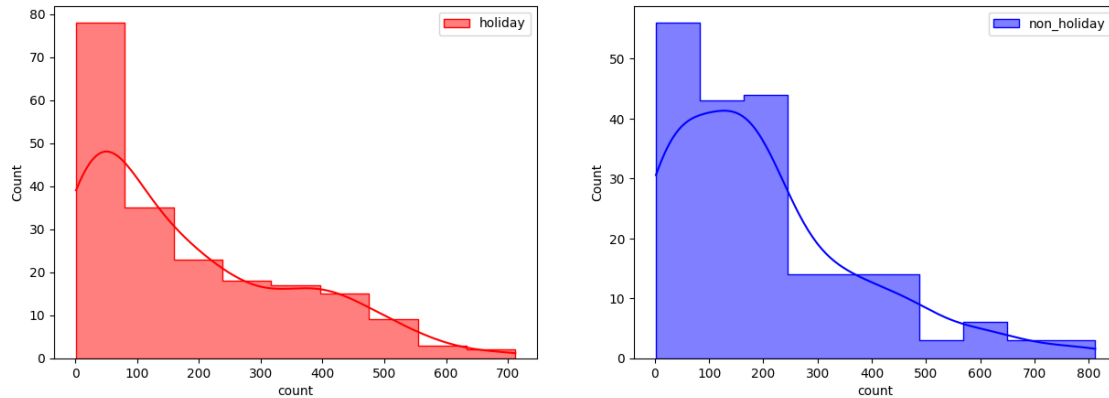
- Based on p-value, we will accept or reject H_0 .

1. $p\text{-val} > \alpha$: Accept H_0
2. $p\text{-val} < \alpha$: Reject H_0

```
[531]: plt.figure(figsize = (15, 5))
plt.subplot(1, 2, 1)
sns.histplot(df.loc[df['holiday'] == 1, 'count'].sample(200),
             element = 'step', color = 'red', kde = True, label = 'holiday')
```

```
plt.legend()
plt.subplot(1, 2, 2)
sns.histplot(df.loc[df['holiday'] == 0, 'count'].sample(200),
             element = 'step', color = 'blue', kde = True, label = 'non_holiday')
plt.legend()
plt.plot()
```

[531]: []

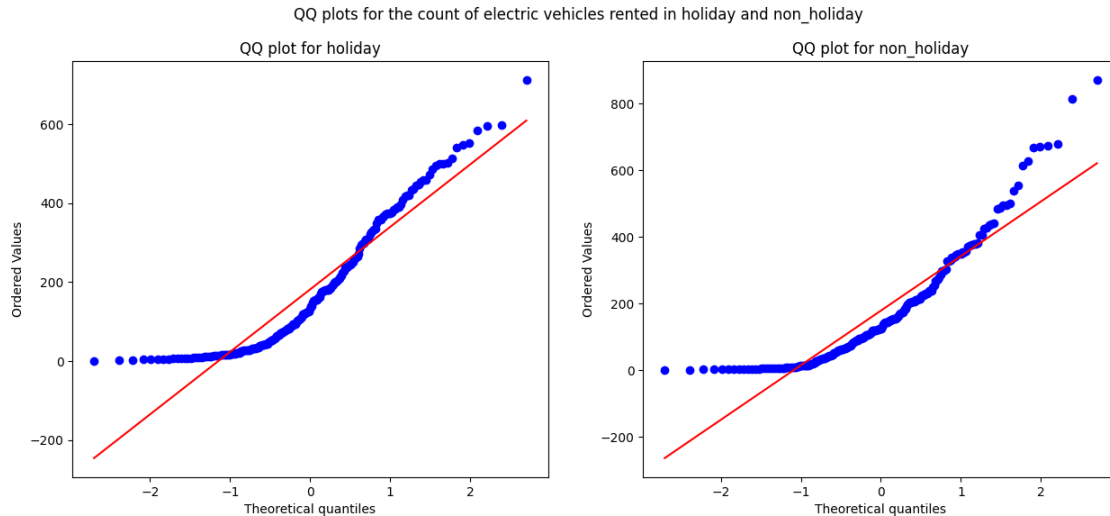


- It can be inferred from the above plot that the distributions do not follow normal distribution.

Distribution check using QQ Plot

```
[532]: plt.figure(figsize = (15, 6))
plt.subplot(1, 2, 1)
plt.suptitle('QQ plots for the count of electric vehicles rented in holiday and non_holiday')
spy.probplot(df.loc[df['holiday'] == 1, 'count'].sample(200), plot = plt, dist = 'norm')
plt.title('QQ plot for holiday')
plt.subplot(1, 2, 2)
spy.probplot(df.loc[df['holiday'] == 0, 'count'].sample(200), plot = plt, dist = 'norm')
plt.title('QQ plot for non_holiday')
plt.plot()
```

[532]: []



- It can be inferred from the above plot that the distributions do not follow normal distribution.

It can be seen from the above plots that the samples do not come from normal distribution.

- Applying Shapiro-Wilk test for normality
- H_0 - The sample follows normal distribution
- H_1 - The sample does not follow normal distribution
- $\alpha = 0.05$
- Test Statistics : Shapiro-Wilk test for normality

```
[533]: test_stat, p_value = spy.shapiro(df.loc[df['holiday'] == 1, 'count'].
        ↪sample(200))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 2.7539268295522817e-11
The sample does not follow normal distribution

```
[534]: test_stat, p_value = spy.shapiro(df.loc[df['holiday'] == 0, 'count'].
        ↪sample(200))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 6.27642112735229e-12
The sample does not follow normal distribution

Transforming the data using boxcox transformation and checking if the transformed data follows normal distribution.

```
[535]: transformed_holiday = spy.boxcox(df.loc[df['holiday'] == 1, 'count'])[0]
test_stat, p_value = spy.shapiro(transformed_holiday)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 2.134933632053098e-07
The sample does not follow normal distribution

```
[536]: transformed_non_holiday = spy.boxcox(df.loc[df['holiday'] == 0, 'count'].
↳sample(5000))[0]
test_stat, p_value = spy.shapiro(transformed_non_holiday)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 5.125840402684802e-26
The sample does not follow normal distribution

- Even after applying the boxcox transformation on each of the “holiday” and “non_holiday” data, the samples do not follow normal distribution.

Homogeneity of Variances using Levene’s test

```
[537]: test_stat, p_value = spy.levene(df.loc[df['holiday'] == 0, 'count'].sample(200),
df.loc[df['holiday'] == 1, 'count'].sample(200))
print('p-value', p_value)
if p_value < 0.05:
    print('The samples do not have Homogenous Variance')
else:
    print('The samples have Homogenous Variance ')
```

p-value 0.32562849834526675
The samples have Homogenous Variance

Since the samples are not normally distributed, T-Test cannot be applied here, we can perform its non parametric equivalent test i.e., Mann-Whitney U rank test for two independent samples.

```
[538]: test_stat, p_value = spy.mannwhitneyu(df.loc[df['holiday'] == 0, 'count'].
↳sample(200),
```



```

df.loc[df['holiday'] == 1, 'count'].
    ↪sample(200))
print('P-value :',p_value)
if p_value < 0.05:
    print('No.of electric cycles rented is not similar for holidays and_
    ↪non-holidays days')
else:
    print('No.of electric cycles rented is similar for holidays and_
    ↪non-holidays')

```

P-value : 0.9087561850082035

No.of electric cycles rented is similar for holidays and non-holidays

Therefore, the number of electric cycles rented is statistically similar for both holidays and non - holidays.

3) Is weather dependent on the season ?

```
[539]: df[['weather', 'season']].describe()
```

```
[539]:
      weather  season
count      10886    10886
unique         4         4
top           1  winter
freq         7192     2734

```

- It is clear from the above statistical description that both 'weather' and 'season' features are categorical in nature.

STEP-1 : Set up Null Hypothesis

1. Null Hypothesis (H_0) - weather is independent of season
2. Alternate Hypothesis (H_A) - weather is dependent of seasons.

STEP-2: Define Test statistics

Since we have two categorical features, the Chi- square test is applicable here. Under H_0 , the test statistic should follow Chi-Square Distribution.

STEP-3: Checking for basic assumptons for the hypothesis (Non-Parametric Test)

1. The data in the cells should be frequencies, or counts of cases.
2. The levels (or categories) of the variables are mutually exclusive. That is, a particular subject fits into one and only one level of each of the variables.
3. There are 2 variables, and both are measured as categories.
4. The value of the cell expecteds should be 5 or more in at least 80% of the cells, and no cell should have an expected of less than one (3).

STEP-4: Compute the p-value and fix value of alpha.

we will be computing the chi square-test p-value using the `chi2_contingency` function using `scipy.stats`. We set our alpha to be 0.05

STEP-5: Compare p-value and alpha.

Based on p-value, we will accept or reject H_0 .

$p\text{-val} > \alpha$: Accept H_0 $p\text{-val} < \alpha$: Reject H_0

The Chi-square statistic is a non-parametric (distribution free) tool designed to analyze group differences when the dependent variable is measured at a nominal level. Like all non-parametric statistics, the Chi-square is robust with respect to the distribution of the data. Specifically, it does not require equality of variances among the study groups or homoscedasticity in the data.

```
[540]: # First, finding the contingency table such that each value is the total number
        ↪ of total bikes rented
        # for a particular season and weather
cross_table = pd.crosstab(index = df['season'],
                          columns = df['weather'],
                          values = df['count'],
                          aggfunc = np.sum).replace(np.nan, 0)

cross_table
```

```
[540]: weather      1      2      3      4
season
fall      470116  139386  31160      0
spring    223009   76406  12919   164
summer    426350  134177  27755      0
winter    356588  157191  30255      0
```

- Since the above contingency table has one column in which the count of the rented electric vehicle is less than 5 in most of the cells, we can remove the weather 4 and then proceed further.

```
[541]: cross_table = pd.crosstab(index = df['season'],
                                columns = df.loc[df['weather'] != 4, 'weather'],
                                values = df['count'],
                                aggfunc = np.sum).to_numpy()[:, :3]

cross_table
```

```
[541]: array([[470116, 139386, 31160],
             [223009,  76406, 12919],
             [426350, 134177, 27755],
             [356588, 157191, 30255]])
```

```
[542]: chi_test_stat, p_value, dof, expected = spy.chi2_contingency(observed =
        ↪ cross_table)
print('Test Statistic =', chi_test_stat)
print('p value =', p_value)
print('-' * 65)
```

```
print("Expected : '\n'", expected)
```

Test Statistic = 10838.372332480214

p value = 0.0

Expected : '

```
' [[453484.88557396 155812.72247031 31364.39195574]
 [221081.86259035 75961.44434981 15290.69305984]
 [416408.3330293 143073.60199337 28800.06497733]
 [385087.91880639 132312.23118651 26633.8500071 ]]
```

Comparing p value with significance level

```
[543]: #Assume signifance level = 5%
alpha = 0.05
if p_value < alpha:
    print('Reject Null Hypothesis')
else:
    print('Failed to reject Null Hypothesis')
```

Reject Null Hypothesis

Therefore, there is statistically significant dependency of weather and season based on the number of number of bikes rented.

4) Is the number of cycles rented is similar or different in different weather ?

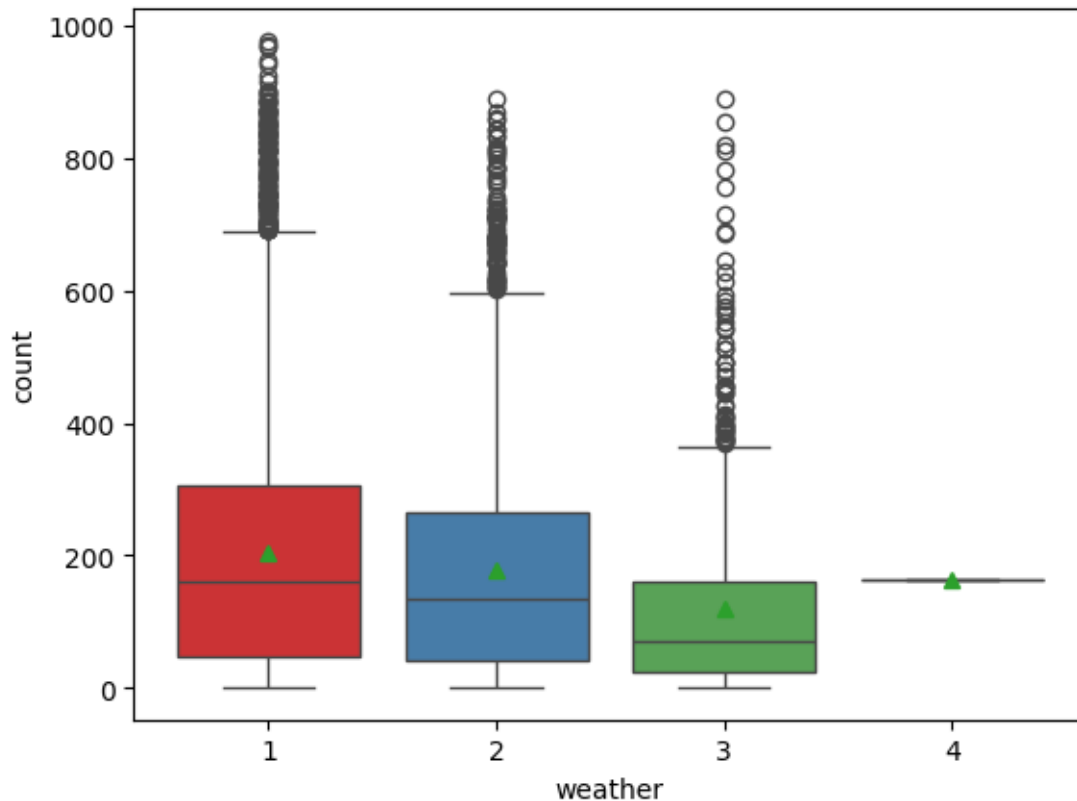
```
[544]: df.groupby(by = 'weather')['count'].describe()
```

```
[544]:
```

	count	mean	std	min	25%	50%	75%	max
weather								
1	7192.0	205.236791	187.959566	1.0	48.0	161.0	305.0	977.0
2	2834.0	178.955540	168.366413	1.0	41.0	134.0	264.0	890.0
3	859.0	118.846333	138.581297	1.0	23.0	71.0	161.0	891.0
4	1.0	164.000000	NaN	164.0	164.0	164.0	164.0	164.0

```
[545]: sns.boxplot(data = df, x = 'weather', y = 'count', showmeans =
↪ True,palette='Set1')
plt.plot()
```

```
[545]: []
```



```
[546]: df_weather1 = df.loc[df['weather'] == 1]
df_weather2 = df.loc[df['weather'] == 2]
df_weather3 = df.loc[df['weather'] == 3]
df_weather4 = df.loc[df['weather'] == 4]
len(df_weather1), len(df_weather2), len(df_weather3), len(df_weather4)
```

[546]: (7192, 2834, 859, 1)

STEP-1 : Set up Null Hypothesis

Null Hypothesis (H_0) - Mean of cycle rented per hour is same for weather 1, 2 and 3. (We wont be considering weather 4 as there in only 1 data point for weather 4 and we cannot perform a ANOVA test with a single data point for a group)

Alternate Hypothesis (H_A) -Mean of cycle rented per hour is not same for season 1,2,3 and 4 are different.

STEP-2 : Checking for basic assumptons for the hypothesis

- Normality check using QQ Plot. If the distribution is not normal, use BOX-COX transform to transform it to normal distribution.
- Homogeneity of Variances using Levene's test
- Each observations are independent.

STEP-3: Define Test statistics

The test statistic for a One-Way ANOVA is denoted as F. For an independent variable with k groups, the F statistic evaluates whether the group means are significantly different.

$$F = \text{MSB} / \text{MSW}$$

Under H_0 , the test statistic should follow F-Distribution.

STEP-4: Decide the kind of test.

We will be performing right tailed f-test

STEP-5: Compute the p-value and fix value of alpha.

we will be computing the anova-test p-value using the `f_oneway` function using `scipy.stats`. We set our alpha to be 0.05

STEP-6: Compare p-value and alpha.

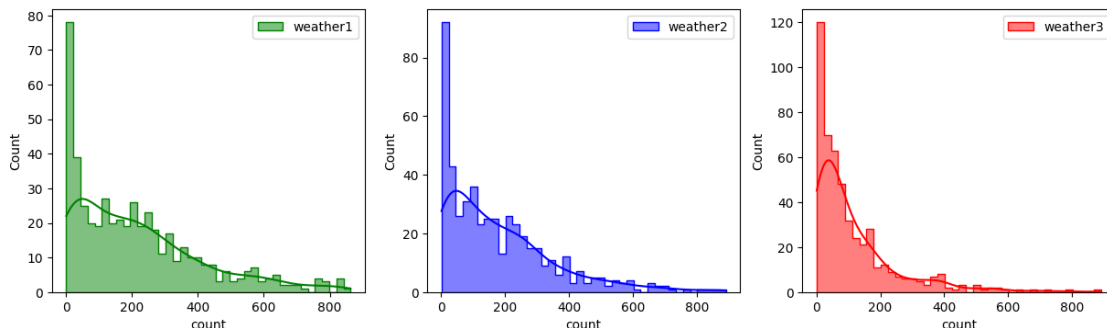
Based on p-value, we will accept or reject H_0 .

p-val > alpha : Accept H_0 p-val < alpha : Reject H_0

Visual Tests to know if the samples follow normal distribution

```
[547]: plt.figure(figsize = (15, 4))
plt.subplot(1, 3, 1)
sns.histplot(df_weather1.loc[:, 'count'].sample(500), bins = 40,
             element = 'step', color = 'green', kde = True, label = 'weather1')
plt.legend()
plt.subplot(1, 3, 2)
sns.histplot(df_weather2.loc[:, 'count'].sample(500), bins = 40,
             element = 'step', color = 'blue', kde = True, label = 'weather2')
plt.legend()
plt.subplot(1, 3, 3)
sns.histplot(df_weather3.loc[:, 'count'].sample(500), bins = 40,
             element = 'step', color = 'red', kde = True, label = 'weather3')
plt.legend()
plt.plot()
```

[547]: []

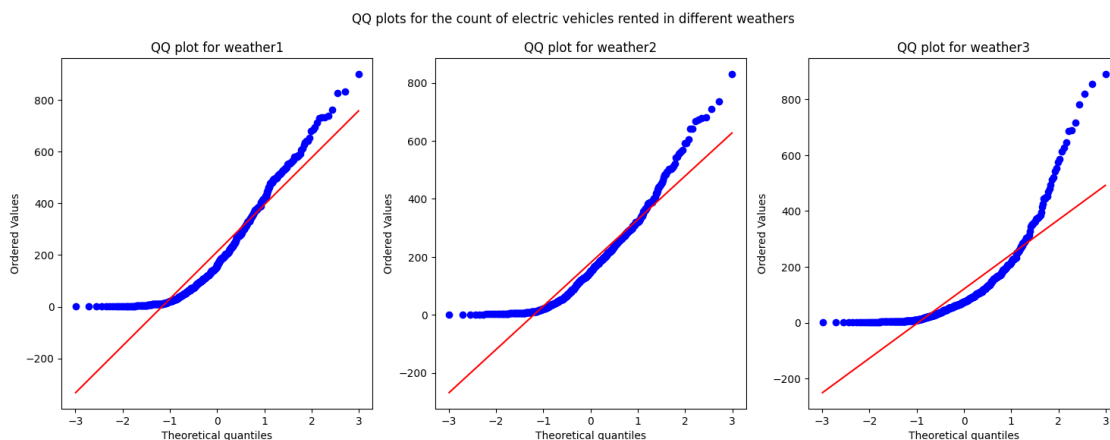


- It can be inferred from the above plot that the distributions do not follow normal distribution.

Distribution check using QQ Plot

```
[548]: plt.figure(figsize = (18, 6))
plt.subplot(1, 3, 1)
plt.suptitle('QQ plots for the count of electric vehicles rented in different_
↳weathers')
spy.probplot(df_weather1.loc[:, 'count'].sample(500), plot = plt, dist = 'norm')
plt.title('QQ plot for weather1')
plt.subplot(1, 3, 2)
spy.probplot(df_weather2.loc[:, 'count'].sample(500), plot = plt, dist = 'norm')
plt.title('QQ plot for weather2')
plt.subplot(1, 3, 3)
spy.probplot(df_weather3.loc[:, 'count'].sample(500), plot = plt, dist = 'norm')
plt.title('QQ plot for weather3')
plt.plot()
```

[548]: []



- It can be inferred from the above plot that the distributions do not follow normal distribution.

It can be seen from the above plots that the samples do not come from normal distribution.

- Applying Shapiro-Wilk test for normality

Ho: The sample follows normal distribution

H1: The sample does not follow normal distribution

- $\alpha = 0.05$
- Test Statistics : Shapiro-Wilk test for normality

```
[549]: test_stat, p_value = spy.shapiro(df_weather1.loc[:, 'count'].sample(500))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 1.1289438913362705e-18
The sample does not follow normal distribution

```
[550]: test_stat, p_value = spy.shapiro(df_weather2.loc[:, 'count'].sample(500))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 9.780524869971015e-22
The sample does not follow normal distribution

Transforming the data using boxcox transformation and checking if the transformed data follows normal distribution.

```
[551]: transformed_weather1 = spy.boxcox(df_weather1.loc[:, 'count'].sample(5000))[0]
test_stat, p_value = spy.shapiro(transformed_weather1)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 2.263031285864687e-27
The sample does not follow normal distribution

```
[552]: transformed_weather2 = spy.boxcox(df_weather2.loc[:, 'count'])[0]
test_stat, p_value = spy.shapiro(transformed_weather2)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 1.9245396515414659e-19
The sample does not follow normal distribution

```
[553]: transformed_weather3 = spy.boxcox(df_weather3.loc[:, 'count'])[0]
test_stat, p_value = spy.shapiro(transformed_weather3)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
```

```
else:
    print('The sample follows normal distribution')
```

p-value 1.4117548371138871e-06

The sample does not follow normal distribution

- Even after applying the boxcox transformation on each of the weather data, the samples do not follow normal distribution.

Homogeneity of Variances using Levene's test

```
[554]: test_stat, p_value = spy.levene(df_weather1.loc[:, 'count'].sample(500),
                                     df_weather2.loc[:, 'count'].sample(500),
                                     df_weather3.loc[:, 'count'].sample(500))

print('p-value', p_value)
if p_value < 0.05:
    print('The samples do not have Homogenous Variance')
else:
    print('The samples have Homogenous Variance ')
```

p-value 1.2060741661463035e-12

The samples do not have Homogenous Variance

Since the samples are not normally distributed and do not have the same variance, f_oneway test cannot be performed here, we can perform its non parametric equivalent test i.e., Kruskal-Wallis H-test for independent samples.

```
[555]: alpha = 0.05
test_stat, p_value = spy.kruskal(df_weather1, df_weather2, df_weather3)
print('Test Statistic =', test_stat)
print('p value =', p_value)
```

Test Statistic = [1.36471292e+01 3.87838808e+01 5.37649760e+00 1.56915686e+01
1.08840000e+04 3.70017441e+01 4.14298489e+01 1.83168690e+03
2.80380482e+01 2.84639685e+02 1.73745440e+02 2.04955668e+02
7.08445555e+01]

p value = [1.08783632e-03 3.78605818e-09 6.79999165e-02 3.91398508e-04
0.00000000e+00 9.22939752e-09 1.00837627e-09 0.00000000e+00
8.15859150e-07 1.55338046e-62 1.86920588e-38 3.12206618e-45
4.13333147e-16]

Comparing p value with significance level

```
[556]: #Assume signifance level = 5%
alpha = 0.05
if p_value.all() < alpha:
    print('Reject Null Hypothesis')
else:
    print('Failed to reject Null Hypothesis')
```

Reject Null Hypothesis

Therefore, the average number of rental bikes is statistically different for different weathers.

5) Is the number of cycles rented is similar or different in different season ?

```
[557]: df.groupby(by = 'season')['count'].describe()
```

```
[557]:
```

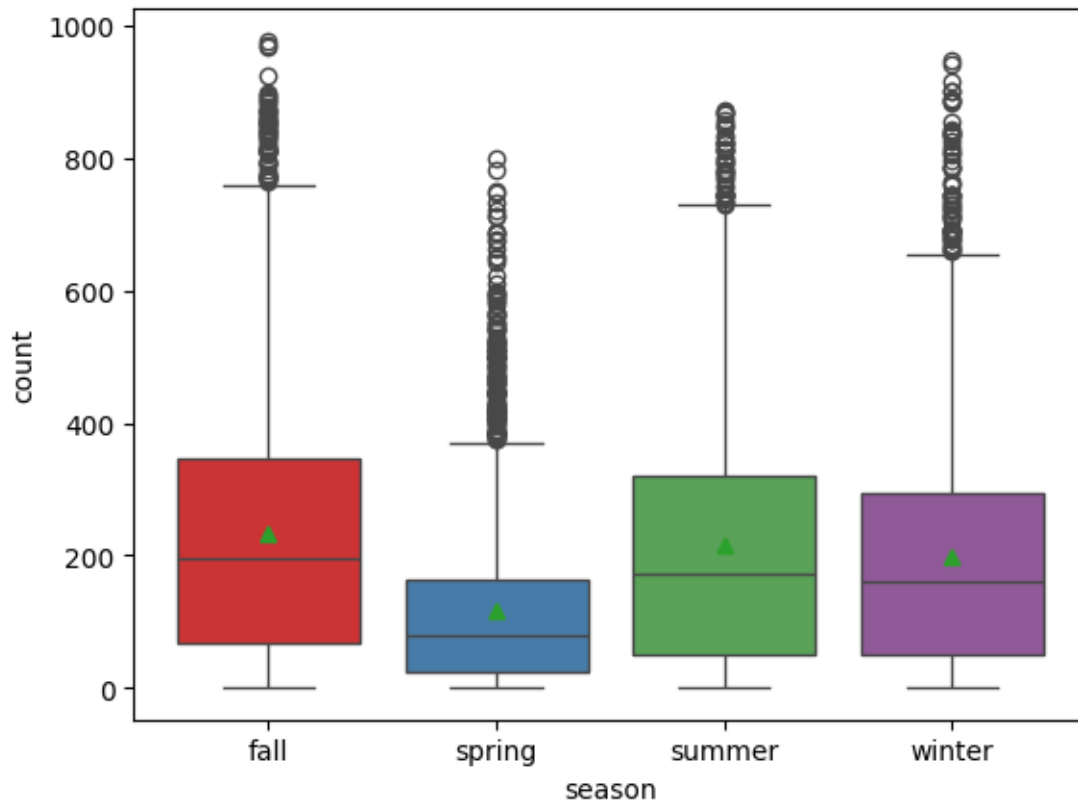
	count	mean	std	min	25%	50%	75%	max
season								
fall	2733.0	234.417124	197.151001	1.0	68.0	195.0	347.0	977.0
spring	2686.0	116.343261	125.273974	1.0	24.0	78.0	164.0	801.0
summer	2733.0	215.251372	192.007843	1.0	49.0	172.0	321.0	873.0
winter	2734.0	198.988296	177.622409	1.0	51.0	161.0	294.0	948.0

```
[558]: df_season_spring = df.loc[df['season'] == 'spring', 'count']
df_season_summer = df.loc[df['season'] == 'summer', 'count']
df_season_fall = df.loc[df['season'] == 'fall', 'count']
df_season_winter = df.loc[df['season'] == 'winter', 'count']
len(df_season_spring), len(df_season_summer), len(df_season_fall),
↳len(df_season_winter)
```

```
[558]: (2686, 2733, 2733, 2734)
```

```
[559]: sns.boxplot(data = df, x = 'season', y = 'count', showmeans =
↳True,palette='Set1')
plt.plot()
```

```
[559]: []
```



STEP-1 : Set up Null Hypothesis

Null Hypothesis (H_0) - Mean of cycle rented per hour is same for season 1,2,3 and 4.

Alternate Hypothesis (H_A) -Mean of cycle rented per hour is different for season 1,2,3 and 4.

STEP-2 : Checking for basic assumptions for the hypothesis

Normality check using QQ Plot. If the distribution is not normal, use BOX-COX transform to transform it to normal distribution.

Homogeneity of Variances using Levene's test

Each observations are independent.

STEP-3: Define Test statistics

The test statistic for a One-Way ANOVA is denoted as F. For an independent variable with k groups, the F statistic evaluates whether the group means are significantly different.

$$F = \text{MSB} / \text{MSW}$$

Under H_0 , the test statistic should follow F-Distribution.

STEP-4: Decide the kind of test.

We will be performing right tailed f-test

STEP-5: Compute the p-value and fix value of alpha.

we will be computing the anova-test p-value using the `f_oneway` function using `scipy.stats`. We set our alpha to be 0.05

STEP-6: Compare p-value and alpha.

Based on p-value, we will accept or reject H_0 . $p\text{-val} > \alpha$: Accept H_0 $p\text{-val} < \alpha$: Reject H_0

The one-way ANOVA compares the means between the groups you are interested in and determines whether any of those means are statistically significantly different from each other.

Specifically, it tests the null hypothesis (H_0):

$$\mu_1 = \mu_2 = \mu_3 = \dots = \mu_k$$

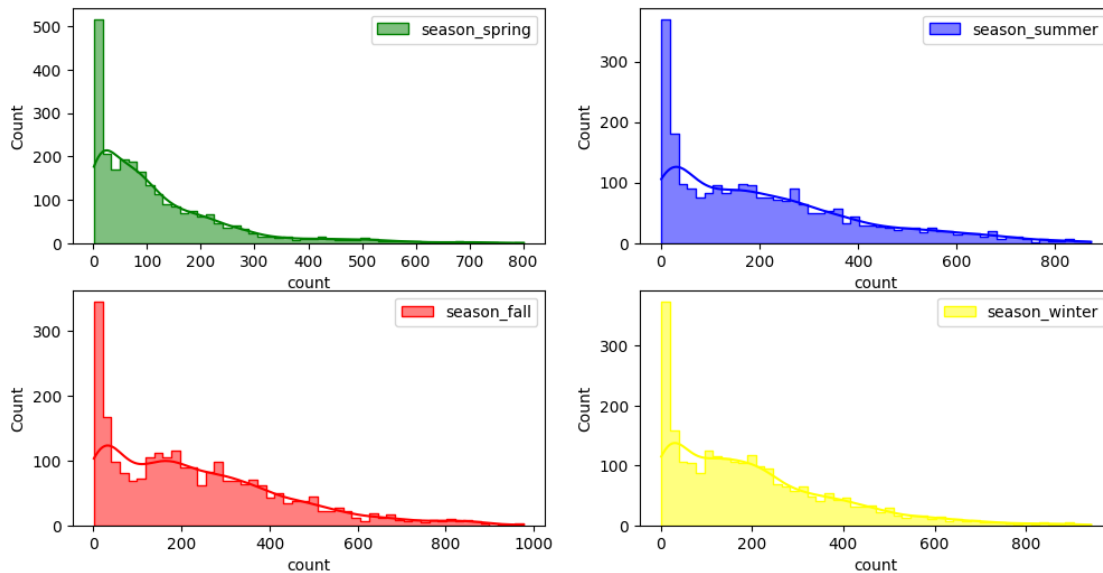
where, μ = group mean and k = number of groups.

If, however, the one-way ANOVA returns a statistically significant result, we accept the alternative hypothesis (H_A), which is that there are at least two group means that are statistically significantly different from each other.

Visual Tests to know if the samples follow normal distribution

```
[560]: plt.figure(figsize = (12, 6))
plt.subplot(2, 2, 1)
sns.histplot(df_season_spring.sample(2500), bins = 50,
             element = 'step', color = 'green', kde = True, label = 'season_spring')
plt.legend()
plt.subplot(2, 2, 2)
sns.histplot(df_season_summer.sample(2500), bins = 50,
             element = 'step', color = 'blue', kde = True, label = 'season_summer')
plt.legend()
plt.subplot(2, 2, 3)
sns.histplot(df_season_fall.sample(2500), bins = 50,
             element = 'step', color = 'red', kde = True, label = 'season_fall')
plt.legend()
plt.subplot(2, 2, 4)
sns.histplot(df_season_winter.sample(2500), bins = 50,
             element = 'step', color = 'yellow', kde = True, label = 'season_winter')
plt.legend()
plt.plot()
```

```
[560]: []
```



- It can be inferred from the above plot that the distributions do not follow normal distribution.

Distribution check using QQ Plot

```
[561]: plt.figure(figsize = (12, 12))
plt.subplot(2, 2, 1)
plt.suptitle('QQ plots for the count of electric vehicles rented in different_
↳seasons')
spy.probplot(df_season_spring.sample(2500), plot = plt, dist = 'norm')
plt.title('QQ plot for spring season')

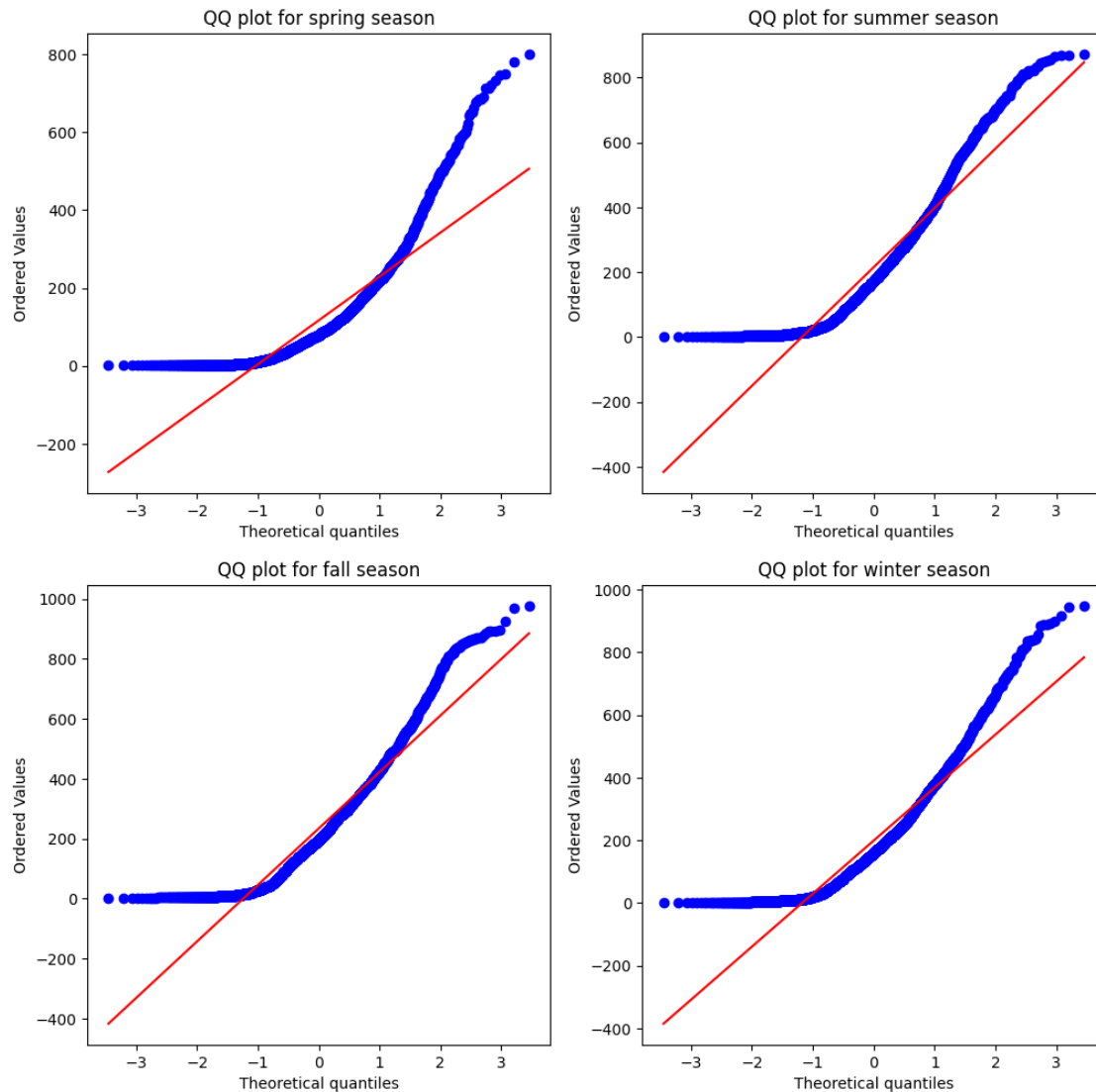
plt.subplot(2, 2, 2)
spy.probplot(df_season_summer.sample(2500), plot = plt, dist = 'norm')
plt.title('QQ plot for summer season')

plt.subplot(2, 2, 3)
spy.probplot(df_season_fall.sample(2500), plot = plt, dist = 'norm')
plt.title('QQ plot for fall season')

plt.subplot(2, 2, 4)
spy.probplot(df_season_winter.sample(2500), plot = plt, dist = 'norm')
plt.title('QQ plot for winter season')
plt.plot()
```

[561]: []

QQ plots for the count of electric vehicles rented in different seasons



- It can be inferred from the above plots that the distributions do not follow normal distribution.

It can be seen from the above plots that the samples do not come from normal distribution.

- Applying Shapiro-Wilk test for normality

Ho: The sample follows normal distribution

H1: The sample does not follow normal distribution

- $\alpha = 0.05$

Test Statistics : Shapiro-Wilk test for normality

```
[562]: test_stat, p_value = spy.shapiro(df_season_spring.sample(2500))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 1.9304125727456351e-47
The sample does not follow normal distribution

```
[563]: test_stat, p_value = spy.shapiro(df_season_summer.sample(2500))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 1.7942344252055064e-37
The sample does not follow normal distribution

```
[564]: test_stat, p_value = spy.shapiro(df_season_fall.sample(2500))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 2.0366191700521773e-35
The sample does not follow normal distribution

```
[565]: test_stat, p_value = spy.shapiro(df_season_winter.sample(2500))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 4.781118900820686e-38
The sample does not follow normal distribution

Transforming the data using boxcox transformation and checking if the transformed data follows normal distribution.

```
[566]: transformed_df_season_spring = spy.boxcox(df_season_spring.sample(2500))[0]
test_stat, p_value = spy.shapiro(transformed_df_season_spring)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 1.421854968262457e-16
The sample does not follow normal distribution

```
[567]: transformed_df_season_summer = spy.boxcox(df_season_summer.sample(2500))[0]
test_stat, p_value = spy.shapiro(transformed_df_season_summer)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 1.726787425084342e-21
The sample does not follow normal distribution

```
[568]: transformed_df_season_fall = spy.boxcox(df_season_fall.sample(2500))[0]
test_stat, p_value = spy.shapiro(transformed_df_season_fall)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 3.822134614655102e-21
The sample does not follow normal distribution

```
[569]: transformed_df_season_winter = spy.boxcox(df_season_winter.sample(2500))[0]
test_stat, p_value = spy.shapiro(transformed_df_season_winter)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 9.538657195837524e-20
The sample does not follow normal distribution

- Even after applying the boxcox transformation on each of the season data, the samples do not follow normal distribution.

Homogeneity of Variances using Levene's test

```
[570]: test_stat, p_value = spy.levene(df_season_spring.sample(2500),
                                     df_season_summer.sample(2500),
                                     df_season_fall.sample(2500),
                                     df_season_winter.sample(2500))
print('p-value', p_value)
if p_value < 0.05:
    print('The samples do not have Homogenous Variance')
else:
    print('The samples have Homogenous Variance ')
```

p-value 5.52019808988135e-109

The samples do not have Homogenous Variance

Since the samples are not normally distributed and do not have the same variance, f_oneway test cannot be performed here, we can perform its non parametric equivalent test i.e., Kruskal-Wallis H-test for independent samples.

```
[571]: alpha = 0.05
test_stat, p_value = spy.kruskal(df_season_spring, df_season_summer,
    ↪df_season_fall,df_season_winter)
print('Test Statistic =', test_stat)
print('p value =', p_value)
```

Test Statistic = 699.6668548181988

p value = 2.479008372608633e-151

omparing p value with significance level

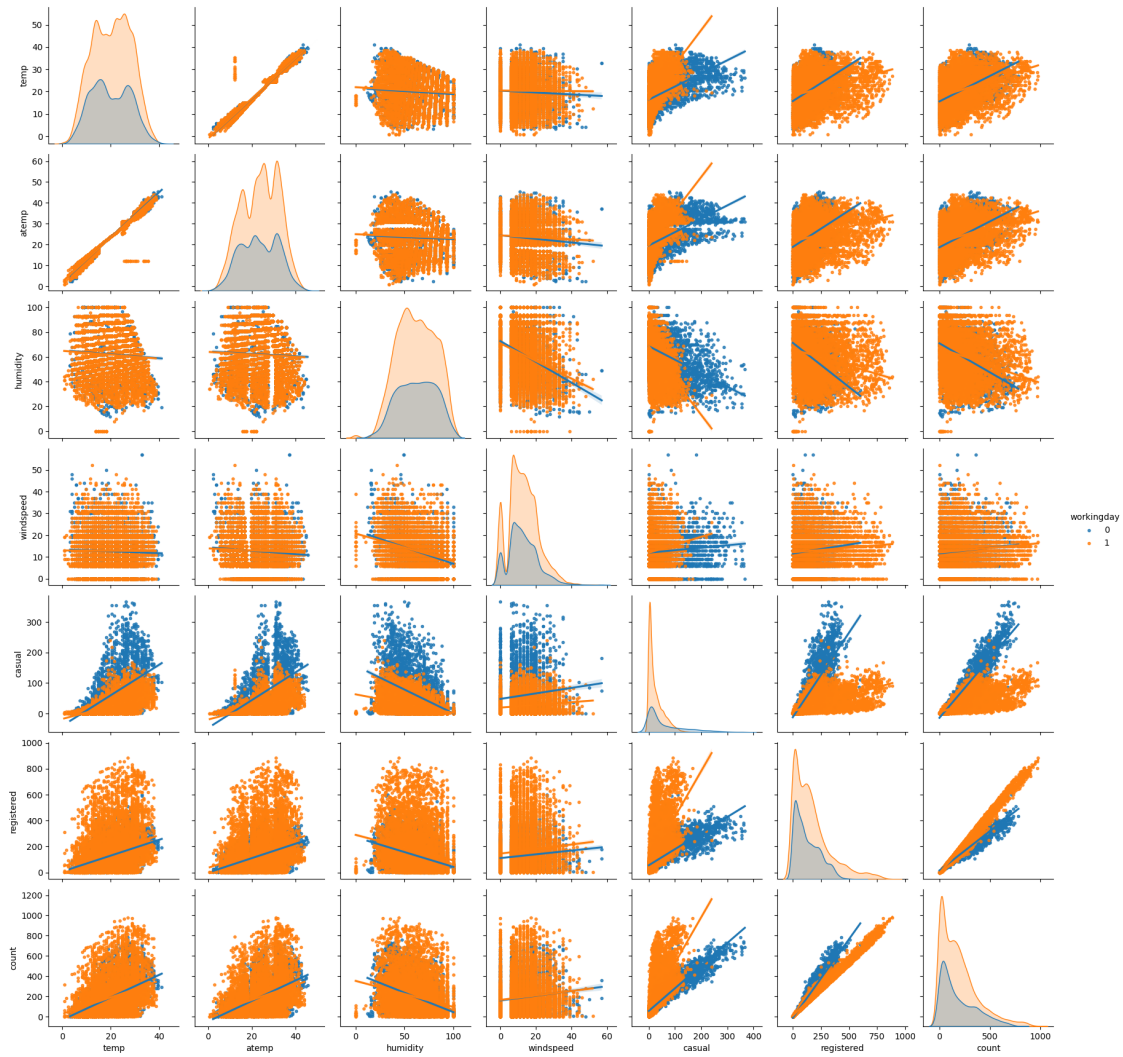
```
[572]: if p_value < alpha:
        print('Reject Null Hypothesis')
    else:
        print('Failed to reject Null Hypothesis')
```

Reject Null Hypothesis

Therefore, the average number of rental bikes is statistically different for different seasons.

```
[573]: sns.pairplot(data = df,
                    kind = 'reg',
                    hue = 'workingday',
                    markers = '.')
plt.plot()
```

```
[573]: []
```

```
[574]: corr_data = df.corr(numeric_only=True)
corr_data
```

```
[574]:
```

	temp	atemp	humidity	windspeed	casual	registered	\
temp	1.000000	0.984948	-0.064949	-0.017852	0.467097	0.318571	
atemp	0.984948	1.000000	-0.043536	-0.057473	0.462067	0.314635	
humidity	-0.064949	-0.043536	1.000000	-0.318607	-0.348187	-0.265458	
windspeed	-0.017852	-0.057473	-0.318607	1.000000	0.092276	0.091052	
casual	0.467097	0.462067	-0.348187	0.092276	1.000000	0.497250	
registered	0.318571	0.314635	-0.265458	0.091052	0.497250	1.000000	
count	0.394454	0.389784	-0.317371	0.101369	0.690414	0.970948	

	count
temp	0.394454
atemp	0.389784

```

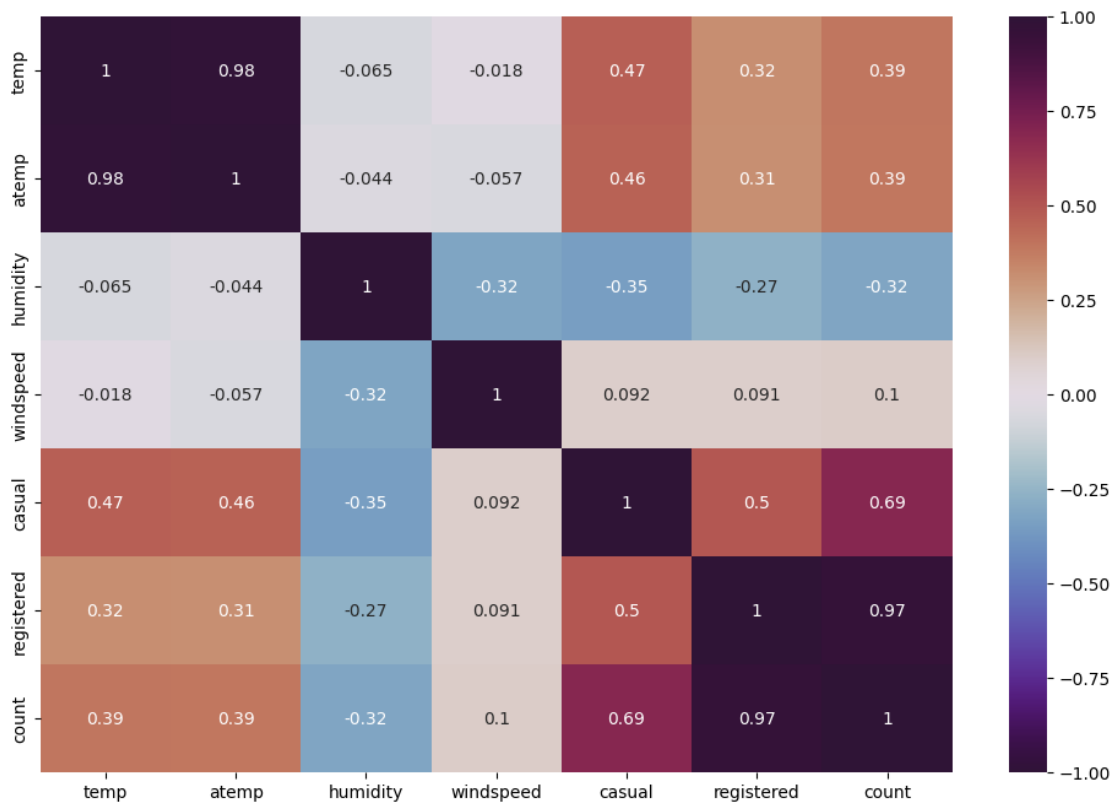
humidity    -0.317371
windspeed   0.101369
casual       0.690414
registered  0.970948
count        1.000000

```

```

[575]: plt.figure(figsize=(12, 8))
sns.heatmap(data=corr_data, cmap='twilight_shifted', annot=True, vmin=-1, v
↪max=1)
plt.show()

```



- Very High Correlation (> 0.9) exists between columns [atemp, temp] and [count, registered]
- High positively / negatively correlation (0.7 - 0.9) does not exist between any columns.
- Moderate positive correlation (0.5 - 0.7) exists between columns [casual, count], [casual, registered].
- Low Positive correlation (0.3 - 0.5) exists between columns [count, temp], [count, atemp], [casual, atemp]
- Negligible correlation exists between all other combinations of columns.

2 Insights

- The provided dataset spans from Timestamp('2011-01-01 00:00:00') to Timestamp('2012-12-19 23:00:00'), encompassing a duration of '718 days 23:00:00'.
- Among every 100 users, approximately 19 are categorized as casual users, while the remaining 81 are registered users.
- The average total hourly bike rental count stands at 144 for the year 2011 and 239 for the year 2012. This indicates an impressive annual growth rate of 65.41% in the demand for electric bikes on an hourly basis.
- An intriguing seasonal pattern emerges, showcasing heightened demand during the spring and summer months, followed by a mild decline in the fall and a more pronounced drop in the winter.
- Notably, the average hourly bike rental count hits its lowest point in January, closely trailed by February and March.
- A distinctive diurnal fluctuation is observed, featuring diminished counts during the early morning hours, a sudden surge in the morning, peak usage during the afternoon, and a gradual tapering off in the evening and nighttime.
- It's worth mentioning that over 80% of the recorded time periods experience temperatures below 28 degrees Celsius.
- Furthermore, humidity levels surpass 40% for more than 80% of the recorded instances, suggesting that humidity levels predominantly oscillate between the optimal range and overly moist conditions.
- In addition, over 85% of the dataset's windspeed measurements fall below 20, indicating generally moderate wind conditions.
- Regarding weather conditions, the highest hourly bike rental counts are observed during clear and cloudy weather, followed by misty conditions and rainy weather. Records for extreme weather conditions are notably scarce.

3 Summary of Hypothesis:

- 1) Is there any effect of Working Day on the number of electric cycles rented ?

The mean hourly count of the total rental bikes is statistically similar for both working and non-working days.

- 2) Is there any effect of holidays on the number of electric cycles rented ?

There is statistically significant dependency of weather and season based on the hourly total number of bikes rented.

- 3) Is weather dependent on the season ?

The hourly total number of rental bikes is statistically different for different weathers.

- 4) Is the number of cycles rented is similar or different in different weather ?

here is no statistically significant dependency of weather 1, 2, 3 on season based on the average hourly total number of bikes rented.

5) Is the number of cycles rented is similar or different in different season ?

The hourly total number of rental bikes is statistically different for different seasons.

4 Recommendations

Strategic Seasonal Marketing: Given the evident seasonal pattern in bike rental counts, Yulu can adapt its marketing strategies strategically. Concentrate on promoting bike rentals during the spring and summer seasons when demand peaks. Consider offering seasonal incentives or exclusive packages to entice more customers during these periods.

Dynamic Time-based Pricing: Leverage the hourly fluctuations in bike rental counts throughout the day. Explore the implementation of dynamic time-based pricing, where rental rates are adjusted to be more affordable during off-peak hours and slightly higher during peak hours. This approach can motivate customers to rent bikes during less congested times, optimizing resource utilization.

Weather-sensitive Promotions: Acknowledge the influence of weather on bike rentals. Create weather-sensitive promotional campaigns targeting customers during clear and partly cloudy conditions, which typically yield the highest rental counts. Yulu can introduce weather-specific discounts to attract more customers during favorable weather conditions.

User-Centric Segmentation: Considering that approximately 81% of users are registered, and the remaining 19% are casual users, Yulu can tailor its marketing and communication strategies with precision. Offer loyalty programs, exclusive incentives, or personalized recommendations to registered users, fostering repeat business. For casual users, emphasize seamless rental experiences and highlight the advantages of bike rentals for occasional use.

Optimized Inventory Management: Analyze demand patterns across different months and fine-tune inventory levels accordingly. During months with lower rental counts such as January, February, and March, Yulu can optimize its inventory to avoid overstocking. Conversely, during peak months, ensure an adequate supply of bikes to meet heightened demand.

Enhanced Weather Data Collection: Given the limited records for extreme weather conditions, consider enhancing data collection procedures for such scenarios. Accumulating more data on extreme weather conditions can facilitate a deeper understanding of customer behavior, enabling adjustments such as offering specialized bike models for different weather conditions or implementing safety measures during extreme weather.

Customer Comfort and Convenience: With generally high humidity levels and temperatures frequently below 28 degrees Celsius, consider enhancing customer comfort. Provide amenities such as umbrellas, rain jackets, or water bottles to elevate the overall biking experience. These thoughtful touches contribute to a positive customer experience and can encourage repeat business.

Collaboration with Weather Services: Explore partnerships with weather services to offer real-time weather updates and forecasts to potential customers. Integrate weather information into marketing campaigns or the rental app to showcase ideal biking conditions, appealing to users who prefer specific weather conditions.

Seasonal Bike Maintenance: Allocate resources for seasonal bike maintenance. Prior to peak seasons, conduct thorough maintenance checks on the bike fleet to ensure optimal performance. Regular inspections and servicing throughout the year can help prevent breakdowns, ensuring customer satisfaction.

Customer Feedback and Reviews: Encourage customers to provide feedback and reviews about their biking experiences. Gathering feedback helps identify areas for improvement, gain insights into customer preferences, and customize services to exceed customer expectations.

Strategic Social Media Marketing: Harness the power of social media platforms to promote Yulu's electric bike rental services strategically. Share captivating visuals of biking experiences in diverse weather conditions, showcase customer testimonials, and engage potential customers through interactive posts and contests. Implement targeted advertising campaigns to reach specific customer segments and boost bookings.

Special Environmental Discounts: Given Yulu's commitment to providing a sustainable solution for vehicular pollution, consider offering special discounts on occasions such as Zero Emissions Day (21st September), Earth Day (22nd April), World Environment Day (5th June), and similar events to attract new environmentally-conscious users.